



**Wydział Matematyki
i Nauk Informatycznych**

POLITECHNIKA WARSZAWSKA

Dokumentacja projektu

*“System rekomendacyjny dla platformy e-commerce
z wykorzystaniem narzędzi Apache i Google Cloud Platform”*

Składowanie danych w systemach Big Data

KGW Gawron

Natalia Choszczyk, Mikołaj Rowicki, Filip Langiewicz

07.01.2026

Spis treści

1. Opis projektu	3
2. Cel biznesowy	3
3. Źródła danych	3
3.1. Dane strumieniowe: aktywność użytkowników na platformie	3
3.2. Dane wsadowe 1: katalog produktów	5
3.3. Dane wsadowe 2: dane użytkowników	5
4. Projekt rozwiązania	6
4.1. Diagram architektury systemu	6
4.2. Przepływy danych w narzędziu Apache Airflow	7
4.3. Apache Kafka – bufor strumieniowy	8
4.4. Apache Hadoop (HDFS) – rozproszony system plików	8
4.5. Apache Hive – hurtownia danych	8
4.6. Apache Spark – silnik analityczny	8
4.7. Apache HBase – warstwa serwisowa	9
5. Środowisko wdrożeniowe	9
5.1. System operacyjny	9
5.2. Zasoby sprzętowe	10
5.3. Konfiguracja sieciowa	10
5.4. Konfiguracja firewall	10
6. Stos technologiczny	11
6.1. Apache Hadoop 3.3.6	11
6.2. Apache Spark 3.5.3	12
6.3. Apache Kafka 3.6.1	12
6.4. Apache Hive 3.1.3 – hurtownia danych	13
6.5. Apache HBase 2.6.4-hadoop3	13
6.6. Apache Airflow 2.8.1	14
6.7. Dodatkowe biblioteki i narzędzia	15
7. Przetwarzanie danych strumieniowych	15
7.1. Symulacja strumienia danych użytkowników	15
7.2. Orkiestracja symulacji przez Apache Airflow	19
7.3. Archiwizacja strumienia do HDFS – Batch Layer	22
7.4. Orkiestracja archiwizacji przez Apache Airflow	27
7.5. Analiza danych strumieniowych w czasie rzeczywistym – Spark Streaming	31
7.6. Orkiestracja Spark Streaming przez Apache Airflow	39
8. Analiza wsadowa Batch Layer	43
8.1. Przepływ danych użytkowników	43
8.2. Przepływ danych produktów	44
8.3. Orkiestracja analiz wsadowych	46
8.4. Analiza lejka zakupowego (Funnel Analysis)	49
8.5. Rekomendacje demograficzne	51
8.6. Analiza efektywności marek	54
8.7. Podsumowanie warstwy wsadowej	56

1. Opis projektu

Celem projektu jest opracowanie prototypowego systemu analitycznego dla platformy e-commerce, łączącego przetwarzanie danych strumieniowych w czasie rzeczywistym z analizą danych wsadowych w celu generowania rekomendacji produktowych i wniosków biznesowych. System wykorzystuje narzędzia ekosystemu Apache (Airflow, Kafka, Spark, Hive, HBase) uruchomione na maszynie wirtualnej w środowisku Google Cloud Platform (GCP), umożliwiając przetwarzanie, analizę i archiwizację danych użytkowników w czasie rzeczywistym.

2. Cel biznesowy

System został zaprojektowany dla średnich i dużych platform e-commerce obsługujących miliony użytkowników i dziesiątki tysięcy produktów. Głównym odbiorcą są działy analityczne, marketingowe oraz zarządy sklepów internetowych, które potrzebują narzędzi do personalizacji doświadczeń zakupowych i optymalizacji sprzedaży.

System rozwiązuje kluczowe problemy biznesowe. Przede wszystkim generuje spersonalizowane rekomendacje produktowe w oparciu o profil demograficzny użytkownika, a także analizuje lejek zakupowy, identyfikując momenty, w których klienci porzucają zakupy, umożliwiając tym samym optymalizację procesu sprzedaży. Ponadto nasze rozwiązanie monitoruje aktualne trendy w czasie rzeczywistym, pozwalając działom marketingu natychmiast reagować na rosnące zainteresowanie konkretnymi produktami i markami.

Architektura Lambda łącząca przetwarzanie strumieniowe z analizami wsadowymi zapewnia równowagę między szybkością reakcji a głębią analiz. Rozwiązanie oparte na technologiach Apache i infrastrukturze Google Cloud Platform jest skalowalne i może rosnąć wraz z platformą e-commerce, stanowiąc fundament dla przyszłych rozszerzeń o algorytmy uczenia maszynowego czy dodatkowe źródła danych.

3. Źródła danych

Projekt wykorzystuje trzy źródła danych opisane poniżej.

3.1. Dane strumieniowe: aktywność użytkowników na platformie

Pierwszym źródłem danych jest strumień zdarzeń opisujący aktywność użytkowników na platformie e-commerce. Obejmuje on interakcje takie jak wyświetlenie produktu, dodanie do koszyka oraz zakup. W naszym rozwiązaniu strumień będzie symulowany poprzez pythonowy skrypt wykonywany przez narzędzie Apache Airflow. Wczytane w ten sposób rekordy z pliku CSV będą sukcesywnie przesyłane do systemu Apache Kafka, co pozwoli na testowanie procesów przetwarzania strumieniowego w warunkach zbliżonych do produkcyjnych.

Źródło danych:

<https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store/data>.

Dane pochodzą z grudnia 2019 r. Rozmiar zbioru wynosi około 9,2 GB (67 542 878 obserwacji). Poniższa tabela przedstawia rozkład zdarzeń według typu aktywności użytkownika:

Typ zdarzenia	Liczba rekordów
view	62 986 067
cart	3 394 763
purchase	1 162 048

Zbiór danych obejmuje również informacje o liczbie unikalnych użytkowników, produktów i kategorii produktowych:

Użytkownicy	Produkty	Kategorie
4 577 232	205 230	1162

Przykładowe rekordy z danych strumieniowych przedstawiono poniżej:

event_time	event_type	product_id	category_id	category_code	brand	price	user_id	user_session
2019-10-01 00:00:00 UTC	view	44600062	2103807459 595387724	NaN	shiseido	35.79	5413121 40	72d76fde-8bb3-4e00 -8c23-a032dfed738c
2019-10-01 00:00:00 UTC	view	3900821	2053013552 326770905	appliances.environment. water_heater	aqua	33.20	5547487 17	9333dfbd-b87a-4708 -9857-6336556b0fcc
2019-10-01 00:00:01 UTC	view	17200506	2053013559 792632471	furniture.living_room.sofa	NaN	543.1 0	5191072 50	566511c2-e2e3-422b -b695-cf8e6e792ca8
2019-10-01 00:00:01 UTC	view	1307067	2053013558 920217191	computers.notebook	lenovo	251.7 4	5500508 54	7c90fc70-0e80-4590 -96f3-13c02c18c713
2019-10-01 00:00:04 UTC	view	1004237	2053013555 631882655	electronics.smartphone	apple	1081. 98	5358712 17	c6bd7419-2748-4c56 -95b4-8cec9ff8b80d

Struktura pliku – opis pól

Pole	Opis
event_time	Czas, kiedy zdarzenie miało miejsce (w UTC).
event_type	Typ zdarzenia.
product_id	ID produktu.
category_id	ID kategorii produktu.

category_code	Kod kategorii produktu (taksonomia), jeśli możliwe było jego ustalenie. Zwykle obecny dla znaczących kategorii, pomijany w przypadku różnego rodzaju akcesoriów.
brand	Nazwa marki zapisana małymi literami. Może być nieobecna.
price	Cena produktu (float). Zawsze obecna.
user_id	Stałe ID użytkownika.
user_session	Tymczasowe ID sesji użytkownika. Takie samo w ramach jednej sesji; zmienia się, gdy użytkownik wraca do sklepu po dłuższej przerwie.

3.2. Dane wsadowe 1: katalog produktów

Drugim źródłem danych jest katalog produktów dostępnych w sklepie internetowym. Dane będą przechowywane w formacie CSV i wygenerowane przez model językowy (LLM) na podstawie informacji zawartych w danych strumieniowych (m.in. kategorii, marek i nazw produktów).

Poniżej przedstawiono przykładowy zestaw danych, które znajdują się w katalogu produktów:

product_id	product_name	category_id	category_name	category_code	brand
1005105	work lights Model 1178	223273209307 7520756	work lights	construction.tool s.light	apple
22700068	fishing equipment Model 101268	223273209164 3068746	fishing equipment		force

3.3. Dane wsadowe 2: dane użytkowników

Trzecim źródłem danych są informacje na temat użytkowników platformy. Zbiór został wygenerowany przy pomocy aplikacji opracowanej w ramach innego projektu akademickiego. Dane mają rozmiar ok. 370 MB, 5 000 000 rekordów i obejmują szczegółowe informacje o użytkownikach.

Przykładowe rekordy z tego zbioru przedstawiono poniżej:

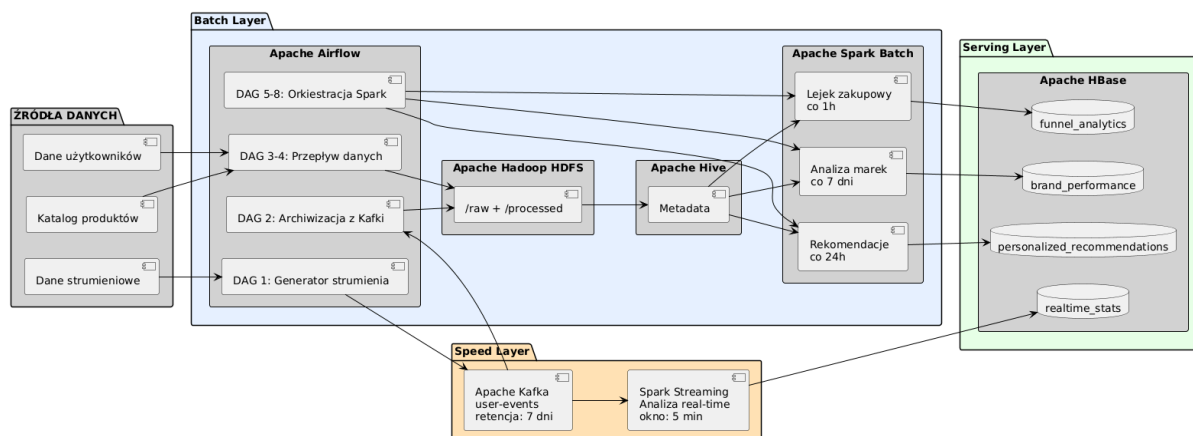
user_id	name	surname	birth_date	city	zip_code	street_name	house_number
29515875	LENA	WITKOWSKA	1987-02-08	Oława	55-201	ul. Elizy Orzeszkowej	69/132
29990697	MARTYNA	WROŃSKA	1974-05-26	Rzeszów	35-202	ul. Leona Staniszewskiego	68
30493659	KINGA	DUDEK	1973-02-19	Warszawa	05-075	ul. Malinowa	50
31198833	PAWEŁ	SZYMCZAK	1999-06-30	Warszawa	02-937	ul. Rumiankowa	31/149
32836036	DAWID	JANIK	1976-11-28	Rzeszów	35-111	ul. Poprzeczna	23
34916060	IRENEUSZ	KAMIŃSKI	1975-10-19	Warszawa	00-177	ul. Błękitna	43

4. Projekt rozwiązania

System analityczny dla platformy e-commerce został zaprojektowany w architekturze Lambda, łączącej przetwarzanie strumieniowe z przetwarzaniem wsadowym oraz warstwą serwisową. Umożliwia to zarówno obsługę zapytań w czasie rzeczywistym, jak i złożonych analiz historycznych.

4.1. Diagram architektury systemu

Poniższy diagram przedstawia architekturę systemu analitycznego z podziałem na trzy główne warstwy zgodnie z modelem Lambda. Speed Layer odpowiada za przetwarzanie strumieniowe w czasie rzeczywistym za pomocą Apache Kafka i Spark Streaming. Batch Layer obejmuje orkiestrację przepływów przez Apache Airflow, przechowywanie danych w rozproszonym systemie plików HDFS, zarządzanie metadanymi przez Apache Hive oraz przetwarzanie wsadowe w Apache Spark. Serving Layer wykorzystuje Apache HBase do udostępniania wstępnie obliczonych wyników analiz. Strzałki na diagramie ilustrują przepływ danych od źródeł przez poszczególne komponenty systemu aż do warstwy serwisowej, z której korzystają aplikacje końcowe.



Rysunek 1: Diagram architektury systemu

4.2. Przepływy danych w narzędziu Apache Airflow

Apache Airflow pełni rolę centralnego orkiestratora wszystkich procesów ETL i analitycznych w systemie. Przepływy pracy są definiowane jako DAG-i, gdzie każdy węzeł reprezentuje konkretne zadanie, a krawędzie określają zależności między nimi.

W projekcie przewidziano następujące przepływy (DAG-i):

1. Przepływ danych strumieniowych

Skrypt wczytuje rekordy z pliku CSV i przesyła je do Kafki w sposób symulujący naturalne tempo aktywności użytkowników na platformie e-commerce, z uwzględnieniem zmiennego obciążenia charakterystycznego dla sklepu internetowego. Skrypt loguje metryki przepustowości i opóźnień.

2. Przepływ danych produktowych

Airflow pobiera plik CSV z katalogiem produktów, przeprowadza walidację danych, konwertuje je do formatu Parquet i tworzy partycjonowanie według kodów kategorii. Przetworzone dane są ładowane do HDFS i rejestrowane w Hive.

3. Przepływ danych użytkowników

Airflow przetwarza dane użytkowników, przeprowadzając transformacje takie jak standaryzację dat, normalizację adresów oraz obliczanie wieku i grupy wiekowej. Dane są konwertowane do formatu Parquet i ładowane do HDFS, a także rejestrowane w Hive.

4. Archiwizacja danych strumieniowych

Uruchamiana cyklicznie co godzinę, konsumuje dane z Kafki i zapisuje je do HDFS w postaci partycjonowanej według daty i godziny, automatycznie rejestrując nowe partycje w Hive.

5. Orkiestracja zadań analitycznych Spark

Orkiestruje zadania analityczne Spark według harmonogramu, zarządza zależnościami między analizami, monitoruje wykonanie i wyzwala aktualizację warstwy serwisowej w HBase po zakończeniu obliczeń.

4.3. Apache Kafka – bufor strumieniowy

Kafka pełni rolę centralnego bufora zdarzeń, zapewniając niezawodny transfer danych strumieniowych. Główny temat user-events zawiera strumień wszystkich zdarzeń użytkowników i jest partycjonowany według identyfikatora użytkownika, zapewniając zachowanie kolejności zdarzeń. Kafka buforuje dane z polityką retencji siedem dni. Dane są regularnie archiwizowane do HDFS przez dedykowany przepływ w Airflow (Przepływ 4), zapewniając długoterminowe przechowywanie przed ich usunięciem z Kafki.

4.4. Apache Hadoop (HDFS) – rozproszony system plików

HDFS oferuje skalowalną pamięć masową z hierarchiczną strukturą katalogów. Katalog raw przechowuje archiwum danych z Kafki podzielone według roku, miesiąca, dnia i godziny oraz katalog produktów i profile użytkowników. Katalog processed zawiera wyniki analiz Spark. Wszystkie dane są w formacie Parquet z kompresją Snappy. Dane zdarzeń są partycjonowane według czasu, a katalog produktów według kodu kategorii. Współczynnik replikacji wynosi dwa dla środowiska deweloperskiego.

4.5. Apache Hive – hurtownia danych

Hive zapewnia warstwę SQL nad danymi w HDFS, zarządzając metadanymi opisującymi strukturę tabel. Tabela user_events jest tabelą zewnętrzną partycjonowaną czasowo, zawierającą wszystkie zdarzenia użytkowników. Tabela product_catalog przechowuje katalog produktów z partycjonowaniem według kodu kategorii. Tabela user_profiles zawiera informacje demograficzne użytkowników oraz kolumny pochodne jak wiek i grupa wiekowa. Hive umożliwia zapytania, zarządza metadanymi i jest zintegrowany ze Spark SQL.

4.6. Apache Spark – silnik analityczny

Spark stanowi rdzeń warstwy przetwarzania, wykonując analizy wsadowe i w czasie rzeczywistym w czterech głównych modułach:

1. Analiza w czasie rzeczywistym

Spark realizuje analizę w czasie rzeczywistym, konsumując dane z Kafki. Oblicza metryki takie jak liczba aktywnych użytkowników, top 10 produktów oraz średnia wartość koszyka w pięciominutowym oknie czasowym, zapisując wyniki do HBase.

2. Analiza “lejka zakupowego”

Uruchamiany codziennie, przetwarza dane z ostatnich 24 godzin. Oblicza, ilu użytkowników wyświetla produkt, następnie dodaje go do koszyka i dokonuje zakupu. Identyfikuje ścieżki użytkowników od wyświetlenia przez koszyk do zakupu, oblicza średni czas między etapami oraz “drop-off rate” z podziałem na kategorie i marki, zapisując wyniki do HBase.

3. Spersonalizowane rekomendacje według profilu demograficznego

Uruchamiany codziennie, łączy dane o zdarzeniach, profilach użytkowników i katalog produktów. Segmentuje użytkowników według wieku i regionu, obliczając top 10 produktów dla każdego segmentu według wyświetleń, i wartości zamówienia. Wyniki są zapisywane do HBase.

4. Analiza efektywności marek

Uruchamiany codziennie, agreguje metryki według marek, identyfikuje trendy oraz bada korelacje z danymi demograficznymi. Wyniki zapisuje do HBase.

4.7. Apache HBase – warstwa serwisowa

HBase przechowuje wstępnie obliczone wyniki analiz Spark, umożliwiając błyskawiczny odczyt z opóźnieniami rzędu milisekund.

Tabela `personalized_recommendations` wykorzystuje identyfikator użytkownika jako klucz i przechowuje spersonalizowane rekomendacje według profilu demograficznego. Tabela `funnel_analytics` używa złożonego klucza `data-kategoria` i zawiera metryki lejka zakupowego. Tabela `realtime_stats` przechowuje metryki w pięciominutowych oknach czasowych. Tabela `brand_performance` zawiera tygodniowe analizy efektywności marek.

5. Środowisko wdrożeniowe

System analityczny został wdrożony na maszynie wirtualnej w środowisku Google Cloud Platform (GCP), co zapewnia skalowalność, elastyczność oraz dostęp do zaawansowanych narzędzi chmurowych. Platforma GCP została wybrana ze względu na możliwość równoległej pracy nad projektem przez kilku członków zespołu. Maszyna wirtualna została skonfigurowana w regionie Europe Central 2 (Warszawa), co minimalizuje opóźnienia dla użytkowników w Polsce.

Instancja VM nosi nazwę `bigdata-vm` i została utworzona w ramach projektu GCP o identyfikatorze `bigdata-ecommerce-kgw`. System działa w strefie dostępności `eu-central-2-b`, co zapewnia wysoką dostępność oraz odporność na awarie w obrębie regionu.

5.1. System operacyjny

Na maszynie wirtualnej został zainstalowany system operacyjny Ubuntu 22.04.5 LTS (Jammy Jellyfish), będący jedną z najpopularniejszych dystrybucji Linuxa dla środowisk big data. System operuje na jądrze Linux w wersji 6.8.0-1045-gcp, które zostało specjalnie zoptymalizowane pod kątem środowiska Google Cloud Platform. Architektura `x86_64` umożliwia pełne wykorzystanie 64-bitowych procesorów oraz efektywne zarządzanie pamięcią powyżej 4 GB. System od momentu uruchomienia działa stabilnie przez ponad dziewięć dni bez przerw, co potwierdza jego niezawodność w warunkach ciągłego przetwarzania danych strumieniowych.

5.2. Zasoby sprzętowe

Maszyna wirtualna została skonfigurowana jako instancja typu n1-standard-4. Typ ten oferuje 4 wirtualne procesory (vCPU) oparte na architekturze Intel Xeon działające z bazową częstotliwością 2.20 GHz. Każdy rdzeń procesora obsługuje technologię hyperthreading, co pozwala na równoległe przetwarzanie dwóch wątków na rdzeń, zwiększając efektywność przy zadaniach wielowątkowych charakterystycznych dla przetwarzania big data.

Konfiguracja pamięci RAM wynosi 14 GB, co wystarcza dla uruchomienia wszystkich komponentów architektury Lambda: Apache Hadoop (HDFS), Apache Kafka, Apache Spark (zarówno streaming jak i batch processing), Apache HBase oraz Apache Airflow. Pamięć została odpowiednio podzielona między poszczególne komponenty, z największym przydziałem dla Spark'a oraz Kafki.

System dysponuje dyskiem trwałym o pojemności 194 GB, co zapewnia wystarczająco dużo przestrzeni dla systemu operacyjnego, aplikacji oraz tymczasowego przechowywania danych. Hadoop HDFS został skonfigurowany z dostępną przestrzenią 193.65 GB. Dyski w GCP są replikowane automatycznie, co zapewnia odporność na awarie sprzętowe.

5.3. Konfiguracja sieciowa

Maszyna wirtualna posiada podwójną konfigurację adresacji IP, umożliwiającą zarówno komunikację wewnętrzną w sieci VPC, jak i dostęp z internetu. Adres IP wewnętrzny wynosi 10.186.0.2 i jest używany do komunikacji między komponentami systemu big data działającymi na tej samej maszynie oraz potencjalnej komunikacji z innymi instancjami w tej samej sieci VPC.

Hostname maszyny został ustawiony jako bigdata-vm, co ułatwia identyfikację instancji w logach systemowych oraz w interfejsach zarządzania GCP. Konfiguracja DNS została automatycznie zarządzana przez Google Cloud, zapewniając rozwiązywanie nazw domenowych zarówno dla usług wewnętrznych GCP, jak i dla zasobów internetowych.

System nasłuchuje na wielu portach sieciowych, udostępniając interfejsy webowe i API poszczególnych komponentów. Port 8080 jest dedykowany dla Apache Airflow (web UI), port 9870 dla Hadoop HDFS NameNode, port 16010 dla HBase Master, port 4040 dla Spark UI (aktywne podczas wykonywania zadań), port 9092 dla Kafka broker oraz port 9090 dla HBase Thrift Server, który umożliwia komunikację z HBase z poziomu języka Python poprzez bibliotekę happybase. Dodatkowo, port 22 jest otwarty dla połączeń SSH, umożliwiając zdalną administrację systemem.

5.4. Konfiguracja firewall

Bezpieczeństwo dostępu do maszyny wirtualnej jest zarządzane przez system firewall Google Cloud Platform, który działa na poziomie sieci VPC. Reguły firewall są definiowane centralnie w konsoli GCP i automatycznie stosowane do odpowiednich instancji. W projekcie bigdata-ecommerce-kgw zostały skonfigurowane następujące reguły firewall:

- default-allow-ssh – zezwala na połączenia SSH (port 22) z dowolnego adresu IP, niezbędne dla zdalnej administracji systemem. Połączenia są chronione kluczami SSH, co zapewnia bezpieczne uwierzytelnianie.
- default-allow-internal – zezwala na cały ruch TCP (porty 0-65535), UDP (porty 0-65535) oraz ICMP między instancjami w tej samej sieci VPC. Reguła ta umożliwia bezproblemową komunikację wewnętrzną w przypadku rozbudowy systemu o dodatkowe węzły.
- default-allow-icmp – zezwala na pakiety ICMP, niezbędne dla narzędzi diagnostycznych takich jak ping i traceroute.
- allow-bigdata-webui – kluczowa reguła utworzona specjalnie dla projektu, otwierająca dostęp do interfejsów webowych komponentów big data. Reguła zezwala na ruch przychodzący (INGRESS) na portach:
 - 8080 (Apache Airflow Web UI)
 - 9870 (Hadoop HDFS NameNode UI)
 - 8088 (YARN ResourceManager UI)

Konfiguracja firewall została wzmocniona po incydencie bezpieczeństwa, podczas którego system został dotknięty atakiem botnetów. Początkowo, w fazie deweloperskiej, niektóre porty usług rozproszonych (w szczególności Zookeeper na porcie 2181) były publicznie dostępne, co doprowadziło do wykrycia nieautoryzowanych prób połączeń z adresów IP należących do sieci botnetowych. W odpowiedzi na ten incydent wszystkie porty usług wewnętrznych (Kafka 9092, Zookeeper 2181, HBase Thrift 9090, HBase Master 16010, Spark UI 4040) zostały ograniczone wyłącznie do komunikacji wewnętrznej lub dostępu przez SSH tunneling.

6. Stos technologiczny

Implementacja systemu analitycznego opiera się na sprawdzonym ekosystemie Apache, który stanowi standard w dziedzinie rozproszonych systemów przetwarzania danych. Wszystkie komponenty zostały zainstalowane w katalogu /opt zgodnie z konwencjami dystrybucji Linux dla oprogramowania zewnętrznego.

6.1. Apache Hadoop 3.3.6

Apache Hadoop stanowi fundament całej infrastruktury, dostarczając rozproszony system plików HDFS (Hadoop Distributed File System) oraz mechanizmy zarządzania zasobami. HDFS przechowuje wszystkie dane projektu w strukturze hierarchicznej, zapewniając odporność na awarie poprzez replikację bloków danych. System operuje w trybie pseudo-rozproszonym, gdzie wszystkie demony Hadoop działają na pojedynczej maszynie. Konfiguracja obejmuje NameNode zarządzający metadanymi systemu plików, DataNode odpowiedzialny za faktyczne przechowywanie bloków danych oraz Secondary NameNode wykonujący okresowe punkty kontrolne metadanych.

Struktura katalogów HDFS została zaprojektowana w następujący sposób. Katalog /raw przechowuje surowe dane strumieniowe archiwizowane z Kafki oraz dane wsadowe (katalog produktów, profile użytkowników). Katalog /processed zawiera wyniki analiz Spark, podzielone na podkatalogi według typu analizy (funnel_analysis, recommendations, brand_performance, realtime_aggregates). Katalog /user/hive/warehouse stanowi domyślną

lokalizację dla tabel zarządzanych przez Hive, podczas gdy /tmp/hive służy jako przestrzeń robocza dla zapytań SQL.

6.2. Apache Spark 3.5.3

Apache Spark 3.5.3 pełni rolę centralnego silnika analitycznego, realizując zarówno przetwarzanie wsadowe w ramach Batch Layer, jak i przetwarzanie strumieniowe w Speed Layer. Wybór tej konkretnej wersji był podyktowany pełną kompatybilnością z Hadoop 3.3.6 oraz dostępnością integracji z Kafka przez Spark Structured Streaming.

Spark został skonfigurowany w trybie standalone, uruchamiając zadania lokalnie na 4 rdzeniach procesora. Dla zadań przetwarzania strumieniowego zaalokowano 4 GB pamięci dla drivera oraz 2 GB dla executorów, co pozwala na efektywne przetwarzanie strumieni danych z Kafki w pięciominutowych oknach czasowych.

Integracja Spark z pozostałymi komponentami realizowana jest poprzez dedykowane biblioteki. Connector Spark-Kafka umożliwia konsumpcję danych strumieniowych z topiku user-events. Spark SQL zapewnia interfejs do odczytu danych z Hive poprzez dostęp do Hive Metastore. Wyniki analiz są zapisywane zarówno do HDFS w formacie Parquet (z kompresją Snappy), jak i bezpośrednio do HBase dla zapytań o niskim opóźnieniu.

Mechanizm "checkpointingu" Spark Structured Streaming został skonfigurowany w katalogu /home/win10/spark-checkpoints/realtime, zapewniając dokładnie jednokrotne przetwarzanie zdarzeń oraz możliwość wznowienia procesów strumieniowych po awarii bez utraty danych.

6.3. Apache Kafka 3.6.1

Apache Kafka 3.6.1 stanowi centralną magistralę zdarzeń, pośrednicząc między źródłami danych strumieniowych a konsumentami. Topik user-events został skonfigurowany z trzema partycjami, co umożliwia równoległe przetwarzanie strumieni oraz zachowanie kolejności zdarzeń w ramach partycji. Partycjonowanie opiera się na identyfikatorze użytkownika, gwarantując, że wszystkie zdarzenia danego użytkownika trafiają do tej samej partycji.

Kafka została skonfigurowana z polityką retencji czternastu dni. Wiadomości są kompresowane algorytmem gzip, redukując zużycie sieci i dysku o około 70-80% w przypadku danych tekstowych w formacie JSON. Pojedynczy broker Kafka obsługuje współczynnik replikacji równy 1.

Zookeeper 3.8.3, dostarczany wraz z dystrybucją Kafki, zarządza metadanymi klastra, koordynacją partycji oraz elekcją liderów. Zookeeper działa na domyślnym porcie 2181, jednak po incydencie z atakiem botnetów dostęp do tego portu został ograniczony wyłącznie do komunikacji wewnętrznej. Kafka broker nasłuchuje na porcie 9092, również zabezpieczonym przed dostępem zewnętrznym.

Przepustowość Kafki w obecnej konfiguracji wynosi około 15 zdarzeń na sekundę, co odpowiada realistycznemu symulowaniu aktywności użytkowników. Taka prędkość pozwala na zachowanie naturalnych wzorców czasowych z danych źródłowych. Kafka buforuje zdarzenia przed ich archiwizacją do HDFS, zapewniając odporność na chwilowe niedostępności konsumentów.

6.4. Apache Hive 3.1.3 – hurtownia danych

Apache Hive 3.1.3 dostarcza warstwę SQL nad danymi przechowywanymi w HDFS, umożliwiając zapytania analityczne bez konieczności pisania kodu MapReduce czy Spark. Hive Metastore zarządza metadanymi opisującymi strukturę tabel, partycje, typy kolumn oraz lokalizację danych w HDFS. Metastore wykorzystuje Apache Derby jako wbudowaną bazę danych.

Tabela `user_events` została zdefiniowana jako tabela zewnętrzna z partycjonowaniem według roku, miesiąca, dnia i godziny. Format przechowywania to Parquet z kompresją Snappy, zapewniający zarówno efektywną kompresję, jak i możliwość kolumnowego odczytu danych.

Tabela `product_catalog` została zdefiniowana jako tabela zewnętrzna przechowująca katalog produktów w formacie Parquet z kompresją Snappy, partycjonowana według kodu kategorii, co umożliwia wydajne zapytania analityczne i selektywny odczyt danych.

Tabela `user_profiles` została zdefiniowana jako tabela zewnętrzna zawierająca profile użytkowników wzbogacone o kolumny pochodne, takie jak wiek oraz grupa wiekowa, obliczane podczas przetwarzania wsadowego w Apache Spark. Dane są przechowywane w formacie Parquet z kompresją Snappy.

Hive automatycznie rejestruje nowe partycje po zakończeniu operacji ładowania danych przez dedykowane przepływy w Airflow.

Integracja Hive ze Spark realizowana jest poprzez Spark SQL, który odczytuje metadane z Hive Metastore i bezpośrednio operuje na plikach Parquet w HDFS, omijając warstwę wykonawczą Hive. To podejście minimalizuje narzut związany z serializacją i deserializacją danych, maksymalizując wydajność zapytań analitycznych. Hive pełni rolę katalogu danych, stanowiąc punkt odniesienia dla wszystkich tabel w systemie.

6.5. Apache HBase 2.6.4-hadoop3

Apache HBase 2.6.4 stanowi rdzeń warstwy serwisowej w architekturze Lambda, zapewniając dostęp do wstępnie obliczonych wyników analiz z opóźnieniem rzędu milisekund. Wybór tej konkretnej wersji był niezwykle istotny, ponieważ wcześniejsze wersje HBase (np. 2.5.7) wykazywały problemy kompatybilności z Hadoop 3.3.6, prowadząc do błędów I/O oraz niestabilności klastra.

HBase działa w trybie rozproszonym (distributed mode), choć wszystkie komponenty (HMaster, RegionServer, Zookeeper) operują na tej samej maszynie. HMaster zarządza metadanymi regionów oraz koordynuje operacje administracyjne takie jak podział regionów czy równoważenie obciążenia. RegionServer odpowiada za faktyczne przechowywanie danych oraz obsługę zapytań odczytu i zapisu. Wbudowany Zookeeper HBase zarządza koordynacją między komponentami, choć w praktyce współdzieli infrastrukturę z Zookeeperem Kafki.

Dane w HBase są przechowywane w HDFS pod ścieżką `/hbase`, co zapewnia trwałość oraz replikację bloków danych zgodnie z polityką HDFS. HBase operuje na modelu klucz-wartość

z obsługą wersjonowania danych, co umożliwia śledzenie historii zmian w metrykach analitycznych.

Szczególnie istotnym komponentem jest HBase Thrift Server działający na porcie 9090, który umożliwia dostęp do HBase z języków innych niż Java. W projekcie wykorzystywana jest biblioteka Python happybase, która komunikuje się z HBase przez protokół Thrift, pozwalając skryptom Spark (PySpark) zapisywać wyniki analiz bezpośrednio do tabel HBase. Thrift Server działa w trybie foreground_start w ramach serwisu systemd, co zapewnia właściwe przechwytywanie sygnałów shutdown.

6.5.1. Zarządzanie komponentami HBase przez systemd

Aby zapewnić wysoką dostępność oraz automatyczne wznowienie komponentów HBase po awarii lub restarcie systemu, utworzono dedykowane serwisy systemd zarządzające trzema kluczowymi procesami HBase: HMaster, RegionServer oraz Thrift Server. Mechanizm systemd zastępuje ręczne uruchamianie skryptów shell, zapewniając spójne zarządzanie cyklem życia usług oraz centralne logowanie przez journald.

Mechanizm systemd zapewnia dodatkowe korzyści operacyjne. Dyrektywa WantedBy=multi-user.target powoduje że serwisy są automatycznie uruchamiane podczas startu systemu, bez konieczności ręcznej interwencji. Polityka RestartSec=10 wprowadza dziesięciosekundowe opóźnienie między kolejnymi próbami restartu, zapobiegając szybkim cyklom restart w przypadku trwałych problemów konfiguracyjnych. Zmienne środowiskowe JAVA_HOME oraz HBASE_HOME są eksportowane bezpośrednio w definicjach serwisów, eliminując zależność od plików profilu użytkownika.

6.6. Apache Airflow 2.8.1

Apache Airflow 2.8.1 pełni rolę centralnego orkiestratora wszystkich procesów ETL oraz zadań analitycznych w systemie. Airflow został zainstalowany w izolowanym środowisku wirtualnym Python (~/airflow-env), co eliminuje konflikty zależności z systemowymi pakietami Python oraz innymi aplikacjami.

Airflow definiuje przepływy pracy jako DAG-i (Directed Acyclic Graphs), gdzie każdy węzeł reprezentuje pojedyncze zadanie, a krawędzie określają zależności czasowe i logiczne między zadaniami. Scheduler monitoruje stan DAG-ów oraz uruchamia zadania zgodnie z harmonogramem, podczas gdy WebServer dostarcza interfejs webowy na porcie 8080 umożliwiający wizualizację przepływów, monitoring wykonań oraz ręczne wyzwalanie zadań.

Baza danych Airflow (SQLite) przechowuje metadane o DAG-ach, historię wykonań, logi oraz konfigurację połączeń do zewnętrznych systemów. Katalog ~/airflow/dags zawiera definicje przepływów w postaci skryptów Python, które są automatycznie wykrywane i ładowane przez Scheduler. Katalog ~/airflow/logs gromadzi szczegółowe logi wykonania każdego zadania, partycjonowane według DAG-a, daty oraz identyfikatora uruchomienia.

Kluczowe przepływy zaimplementowane w Airflow obejmują: (1) symulację strumienia zdarzeń użytkowników, która wczytuje dane CSV i publikuje je do Kafki w kontrolowanym tempie; (2) ładowanie katalogu produktów oraz profili użytkowników do HDFS z transformacjami ETL; (3) cykliczną archiwizację danych z Kafki do HDFS

z partycjonowaniem czasowym; (4) orkiestrację zadań analitycznych Spark zgodnie z harmonogramem.

6.7. Dodatkowe biblioteki i narzędzia

Oprócz głównych komponentów Apache, system wykorzystuje szereg bibliotek wspomagających integrację oraz przetwarzanie danych. Biblioteka happybase (Python) zapewnia interfejs do komunikacji z HBase poprzez Thrift Server, umożliwiając skryptom PySpark zapisywanie wyników analiz bez konieczności używania natywnego API Java. Biblioteka kafka-python służy do implementacji producenta Kafki w symulatorze strumieni danych, oferując wysokopoziomowy interfejs do publikowania wiadomości.

Biblioteki pandas oraz pyarrow są wykorzystywane do manipulacji danymi w formacie DataFrame oraz konwersji do formatu Parquet, który stanowi standard przechowywania danych w HDFS. PySpark natywnie wspiera format Parquet poprzez DataFrame API, co umożliwia bezproblemową integrację z Hive oraz efektywne zapytania kolumnowe. Kompresja Snappy, zintegrowana z Parquet, zapewnia szybką kompresję i dekompresję przy akceptowalnym współczynniku kompresji.

Java jest kluczowa dla wszystkich komponentów Apache, które są zbudowane na Java Virtual Machine. Zmienne środowiskowe JAVA_HOME, HADOOP_HOME, SPARK_HOME, KAFKA_HOME, HIVE_HOME oraz HBASE_HOME są konsekwentnie eksportowane w pliku ~/.bashrc, zapewniając spójną konfigurację.

7. Przetwarzanie danych strumieniowych

Speed Layer w architekturze Lambda odpowiada za przetwarzanie danych w czasie rzeczywistym, umożliwiając natychmiastowe reakcje na zachowania użytkowników platformy e-commerce. Warstwa ta realizuje trzy kluczowe funkcjonalności: symulację strumienia zdarzeń użytkowników, archiwizację danych do HDFS oraz analizę strumieniową z wykorzystaniem Spark Structured Streaming. Wszystkie te procesy są orkiestrowane przez Apache Airflow, zapewniając niezawodność, monitoring oraz automatyczne wznowienie po awariach.

7.1. Symulacja strumienia danych użytkowników

7.1.1. Architektura symulatora

Symulacja strumienia zdarzeń realizowana jest przez dedykowany skrypt pythonowy kafka_realtime_simulator.py, który odtwarza aktywność użytkowników na podstawie historycznego zbioru danych z grudnia 2019 roku. Symulator implementuje zaawansowane mapowanie czasowe, przekształcając daty z 2019 roku na bieżący miesiąc (grudzień 2025 lub styczeń 2026), co pozwala na realistyczne testowanie systemu w czasie rzeczywistym bez konieczności oczekiwania na faktyczne zdarzenia.

Klasa RealtimeEcommerceSimulator stanowi rdzeń implementacji, zarządzając całym cyklem życia strumienia od wczytania danych, przez mapowanie czasowe, aż po publikację do Apache Kafka. Inicjalizacja symulatora obejmuje utworzenie producenta Kafka

z konfiguracją zoptymalizowaną pod kątem przepustowości i niezawodności. Kompresja gzip zmniejsza transfer sieciowy o około 70-80%, batch size 16KB agreguje wiadomości przed wysłaniem, a linger time 10ms wprowadza minimalne opóźnienie.

Kolejka priorytetowa stanowi centralną strukturę danych symulatora, przechowując zdarzenia uporządkowane według zmapowanego czasu wystąpienia. Wykorzystanie licznika `itertools.count()` jako drugiego elementu klucza sortowania rozwiązuje problem porównywalności zdarzeń o identycznych stemplach czasowych, eliminując błędy `TypeError`, które występowały we wcześniejszych wersjach implementacji. Mechanizm kolejki priorytetowej zapewnia że zdarzenia są publikowane do Kafki w ścisłej kolejności chronologicznej, co jest kluczowe dla zachowania spójności analiz.

7.1.2. Mapowanie czasowe zdarzeń

Algorytm mapowania czasowego implementowany przez metodę `map_december_to_current()` realizuje inteligentne przekształcenie dat z grudnia 2019 roku na bieżący kontekst czasowy. Dni 27-31 grudnia 2019 są mapowane na odpowiadające im dni grudnia 2025, podczas gdy dni 1-26 grudnia 2019 są mapowane na styczeń 2026.

Proces mapowania zachowuje precyzję do sekundy, kopiując godzinę, minutę i sekundę z oryginalnego zdarzenia. Dzięki temu wzorce czasowe w danych źródłowych (takie jak godziny szczytu aktywności, sezonowość w ciągu dnia) są w pełni odwzorowane w strumieniu symulowanym. Każde zdarzenie wzbogacane jest o trzy znaczniki czasowe: `original_time` zawierający oryginalny stempel czasowy z 2019 roku, `mapped_time` zawierający zmapowany stempel czasowy na 2025/2026, oraz `processing_time` zawierający faktyczny czas publikacji do Kafki, co umożliwia analizę opóźnień przetwarzania.

Mechanizm `should_send_now()` sprawdza, czy zmapowany czas zdarzenia już minął względem bieżącego czasu systemowego. Jeśli zdarzenie jest przyszłe, symulator wstrzymuje publikację i umieszcza zdarzenie z powrotem w kolejce priorytetowej. Jeśli opóźnienie jest krótsze niż godzina, symulator aktywnie czeka na odpowiedni moment publikacji. To podejście zapewnia, że strumień danych odtwarza naturalne tempo aktywności użytkowników, zamiast publikować wszystkie zdarzenia natychmiast z maksymalną prędkością.

7.1.3. Publikacja do Apache Kafka

Każde zdarzenie jest publikowane do topiku `user-events` z kluczem partycji opartym na identyfikatorze użytkownika. Kafka automatycznie dystrybuje zdarzenia między trzy partycje topiku na podstawie hasha klucza, zapewniając, że wszystkie zdarzenia danego użytkownika trafiają do tej samej partycji. To gwarantuje zachowanie kolejności zdarzeń w obrębie pojedynczej sesji użytkownika, co jest krytyczne dla analiz ścieżek zakupowych. Rysunek 2 przedstawia offsety we wszystkich trzech partycjach topiku `user-events` w momencie przeprowadzania testu, demonstrując równomierne rozłożenie obciążenia oraz całkowitą liczbę zdarzeń przetworzonych przez system.


```
bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ kafka-run-class.sh kafka.tools.GetOffsetShell \
--broker-list localhost:9092 \
--topic user-events
user-events:0:2358921
user-events:1:2358723
user-events:2:2348634
```

Rysunek 2: Liczba eventów zgromadzonych w Kafce na dzień 30.12.2025 r. (godzina 19:00)

Format wiadomości to JSON zawierający wszystkie pola ze źródłowego CSV: typ zdarzenia (event_type), identyfikator produktu (product_id), kategorię (category_code), markę (brand), cenę (price), identyfikator użytkownika (user_id) oraz identyfikator sesji (user_session). Pola puste w oryginalnych danych (reprezentowane jako 'NaN' lub pusty string) są normalizowane do wartości null w JSON, co ułatwia późniejsze przetwarzanie w Spark. Stempel czasowy processing_time dodawany w momencie publikacji umożliwia monitoring opóźnień systemu. Rysunek 3 pokazuje trzy reprezentatywne zdarzenia JSON pobrane bezpośrednio z topiku Kafki, ilustrując strukturę danych oraz obecność wszystkich trzech znaczników czasowych: original_time, mapped_time oraz processing_time.

```
bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ timeout 5 kafka-console-consumer.sh \
--bootstrap-server localhost:9092 \
--topic user-events \
--max-messages 3 \
--from-beginning
{"event_time": "2019-12-27 00:00:01 UTC", "event_type": "view", "product_id": 6200046, "category_id": "2232732091961835858", "category_code": "appliances.environment.air_heater", "brand": "almacom", "price": 216.22, "user_id": 514029317, "user_session": "01cf396e-03f1-4ed0-ad3d-1142ad8d008f", "original_time": "2019-12-27 00:00:01 UTC", "mapped_time": "2025-12-27T00:00:01", "processing_time": "2025-12-27T00:20:24.551990"}
{"event_time": "2019-12-27 00:00:01 UTC", "event_type": "view", "product_id": 14701293, "category_id": "2053013552570040549", "category_code": "electronics.video.projector", "brand": "sv", "price": 411.59, "user_id": 529004056, "user_session": "afe756cb-50c8-4a0f-bc28-212f8ff361c5", "original_time": "2019-12-27 00:00:01 UTC", "mapped_time": "2025-12-27T00:00:01", "processing_time": "2025-12-27T00:20:24.553059"}
{"event_time": "2019-12-27 00:00:02 UTC", "event_type": "view", "product_id": 100036212, "category_id": "2053013559792632471", "category_code": "furniture.living room.sofa", "brand": "pinskdrev", "price": 386.11, "user_id": 588256563, "user_session": "c4c488bf-3af7-4fdf-9fc8-7084721e2091", "original_time": "2019-12-27 00:00:02 UTC", "mapped_time": "2025-12-27T00:00:02", "processing_time": "2025-12-27T00:20:24.553984"}
Processed a total of 3 messages
```

Rysunek 3: Przykładowe 3 eventy zbuforowane w Kafce

Symulator implementuje mechanizm agregacji metryk, logując postęp co 1000 wysłanych zdarzeń. Statystyki obejmują liczbę załadowanych zdarzeń z CSV, liczbę faktycznie opublikowanych, liczbę błędów oraz średnią przepustowość w zdarzeniach na sekundę. Rozkład zdarzeń według godziny pozwala na weryfikację zachowania wzorców czasowych. Okresowe wywołanie producer.flush() co 10 sekund wymusza natychmiastowe przesłanie wszystkich zbuforowanych wiadomości do Kafki, minimalizując ryzyko utraty danych w przypadku nagłego zakończenia procesu.

7.1.4. Zarządzanie cyklem życia symulatora

Symulator tworzy plik PID w lokalizacji /tmp/realtime_simulator_YYYY-MM-DD.pid, gdzie data odpowiada symulowanemu dniu. Plik ten służy do identyfikacji działających instancji symulatora oraz zapobiega uruchamianiu wielokrotnych symulatorów dla tej samej daty. Mechanizm sprawdzania istnienia pliku PID oraz walidacji, czy proces o danym identyfikatorze faktycznie działa, eliminuje problemy z "zombie" plikami PID pozostawionymi po nieprawidłowym zakończeniu procesu. Rysunek 4 pokazuje listę plików PID dla różnych dat symulacji, gdzie każdy plik odpowiada jednemu dniu grudnia 2025.

```
bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ ls -lh /tmp/realtime_simulator *.pid
-rw----- 1 bigdatacombains bigdatacombains 7 Dec 28 00:03 /tmp/realtime_simulator_2025-12-27.pid
-rw----- 1 bigdatacombains bigdatacombains 7 Dec 29 00:03 /tmp/realtime_simulator_2025-12-28.pid
-rw----- 1 bigdatacombains bigdatacombains 7 Dec 29 19:26 /tmp/realtime_simulator_2025-12-29.pid
-rw----- 1 bigdatacombains bigdatacombains 7 Dec 30 03:21 /tmp/realtime_simulator_2025-12-30.pid
```

Rysunek 4: Lista plików PID utworzonych przez symulatory (stan na 30.12.2025 r.)

Logowanie realizowane jest na dwa strumienie jednocześnie: plik /home/win10/airflow/logs/kafka_realtime_simulator.log oraz standardowe wyjście konsoli. Umożliwia to zarówno monitoring w czasie rzeczywistym przez Airflow, jak i późniejszą analizę historyczną poprzez przegląd plików log. Format logów zawiera stempel czasowy, poziom ważności (INFO/WARNING/ERROR) oraz szczegółowy komunikat, co ułatwia debugowanie problemów produkcyjnych. Rysunek 5 prezentuje typowy output symulatora, pokazujący postęp ładowania zdarzeń.

```
bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ tail -50 /home/win10/airflow/logs/kafka_realtime_simula
tor.log
2025-12-30 16:57:59,002 - INFO - Postęp: wysłano 2007000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 16:58:24,003 - INFO - Postęp: wysłano 2008000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 16:58:50,009 - INFO - Postęp: wysłano 2009000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 16:59:16,003 - INFO - Postęp: wysłano 2010000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 16:59:42,004 - INFO - Postęp: wysłano 2011000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 17:00:07,004 - INFO - Postęp: wysłano 2012000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 17:00:33,003 - INFO - Postęp: wysłano 2013000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 17:00:58,005 - INFO - Postęp: wysłano 2014000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 17:01:23,004 - INFO - Postęp: wysłano 2015000/2495654 eventów, błędy: 0, prędkość: 40.99 event/s
2025-12-30 17:01:47,011 - INFO - Postęp: wysłano 2016000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:02:14,006 - INFO - Postęp: wysłano 2017000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:02:39,011 - INFO - Postęp: wysłano 2018000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:03:03,002 - INFO - Postęp: wysłano 2019000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:03:28,008 - INFO - Postęp: wysłano 2020000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:03:54,002 - INFO - Postęp: wysłano 2021000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:04:20,003 - INFO - Postęp: wysłano 2022000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:04:46,007 - INFO - Postęp: wysłano 2023000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:05:12,002 - INFO - Postęp: wysłano 2024000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:05:38,002 - INFO - Postęp: wysłano 2025000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:06:03,004 - INFO - Postęp: wysłano 2026000/2495654 eventów, błędy: 0, prędkość: 40.98 event/s
2025-12-30 17:06:29,004 - INFO - Postęp: wysłano 2027000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:06:55,004 - INFO - Postęp: wysłano 2028000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:07:21,004 - INFO - Postęp: wysłano 2029000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:07:47,004 - INFO - Postęp: wysłano 2030000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:08:12,012 - INFO - Postęp: wysłano 2031000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:08:38,007 - INFO - Postęp: wysłano 2032000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:09:04,011 - INFO - Postęp: wysłano 2033000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:09:29,007 - INFO - Postęp: wysłano 2034000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:09:55,003 - INFO - Postęp: wysłano 2035000/2495654 eventów, błędy: 0, prędkość: 40.97 event/s
2025-12-30 17:10:22,008 - INFO - Postęp: wysłano 2036000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:10:48,009 - INFO - Postęp: wysłano 2037000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:11:14,002 - INFO - Postęp: wysłano 2038000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:11:39,006 - INFO - Postęp: wysłano 2039000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:12:06,001 - INFO - Postęp: wysłano 2040000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:12:33,001 - INFO - Postęp: wysłano 2041000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:12:58,004 - INFO - Postęp: wysłano 2042000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:13:23,005 - INFO - Postęp: wysłano 2043000/2495654 eventów, błędy: 0, prędkość: 40.96 event/s
2025-12-30 17:13:49,004 - INFO - Postęp: wysłano 2044000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:14:14,002 - INFO - Postęp: wysłano 2045000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:14:39,002 - INFO - Postęp: wysłano 2046000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:15:03,003 - INFO - Postęp: wysłano 2047000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:15:28,006 - INFO - Postęp: wysłano 2048000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:15:55,002 - INFO - Postęp: wysłano 2049000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
2025-12-30 17:16:22,003 - INFO - Postęp: wysłano 2050000/2495654 eventów, błędy: 0, prędkość: 40.95 event/s
```

Rysunek 5: Logi symulatora

Rysunek 6 przedstawia szczegółowe metryki działającego procesu, w tym zużycie pamięci RAM (13.9%, ~2GB), zużycie CPU (2.0%) oraz czas działania (elapsed time), potwierdzając długoterminową stabilność symulatora działającego nieprzerwanie przez ponad 14 godzin.


```
bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ ps -p $PID -o pid,ppid,%mem,%cpu,etime,cmd
PID      PPID %MEM %CPU   ELAPSED CMD
814670    1 13.9  2.0   14:07:11 python3 kafka_realtime_simulator.py /home/win10/2019-Dec.csv 2025-12-3
```

Rysunek 6: Statystyki symulatora

7.2. Orkiestracja symulacji przez Apache Airflow

7.2.1. Struktura DAG realtime_ecommerce_stream

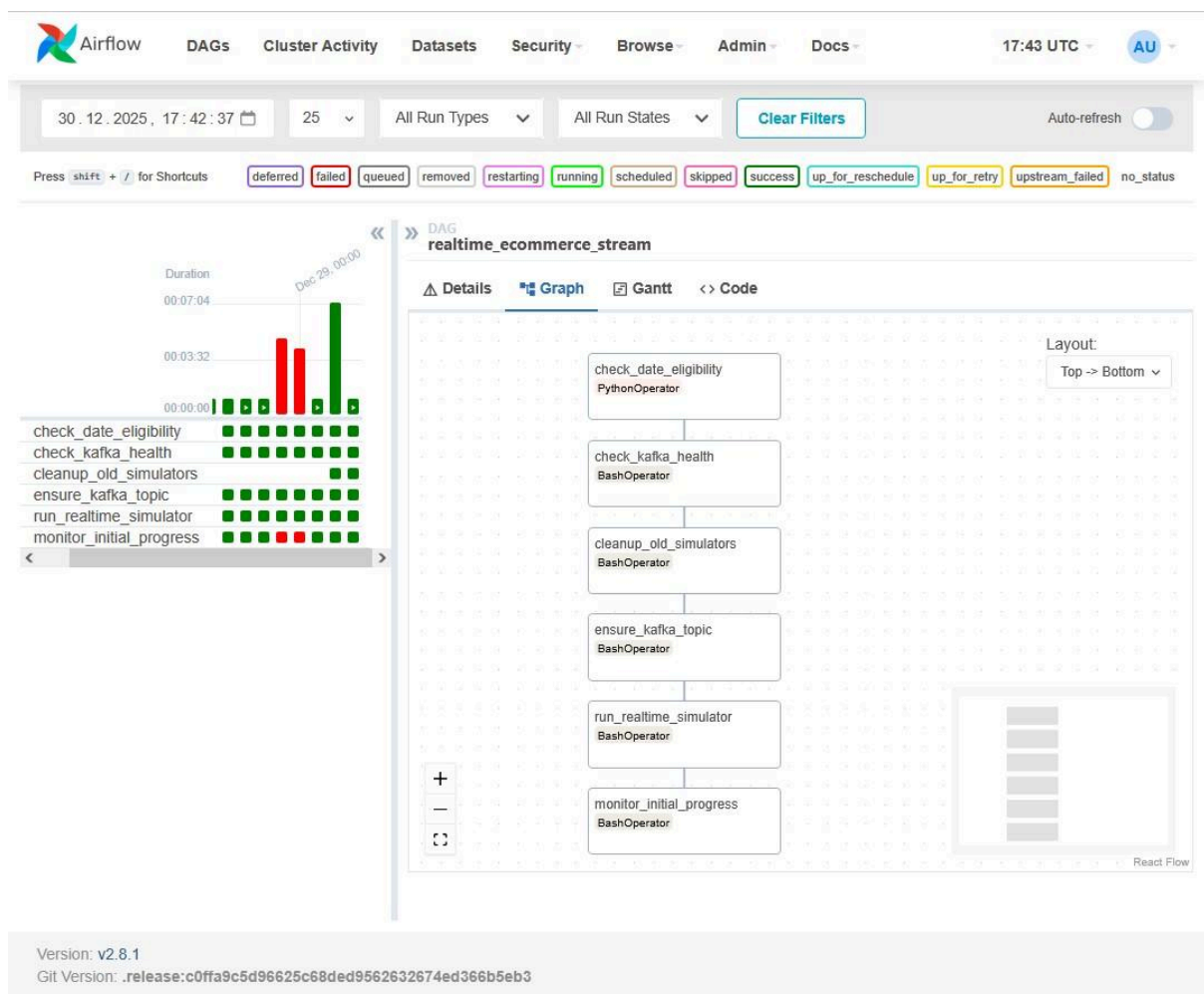
DAG o identyfikatorze realtime_ecommerce_stream odpowiada za codzienne uruchamianie symulatora strumienia danych zgodnie z harmonogramem @daily, czyli o północy czasu lokalnego. Rysunek 7 prezentuje szczegółowe właściwości DAG-u, potwierdzając kluczowe parametry konfiguracyjne. Własność max_active_runs: 1 zapewnia, że w danym momencie działa maksymalnie jedna instancja DAG (zapobiegając uruchamianiu wielokrotnych symulatorów dla tej samej daty), zaś własność is_paused: False wskazuje, że DAG jest aktywny i scheduler Airflow może go wykonywać zgodnie z harmonogramem. Właściwości next_dagrun: 2025-12-30T00:00:00 oraz next_dagrun_data_interval (wskazująca na przedział 30-31 grudnia) potwierdzają że następne zaplanowane wykonanie nastąpi o północy 31 grudnia dla symulacji dnia 30 grudnia. Flaga catchup=False (niewidoczna w outputcie, ale obecna w kodzie DAG) wyłącza historyczne odtwarzanie wykonania dla dni przed datą aktywacji DAG, co jest pożądane w kontekście symulacji czasu rzeczywistego.

```
(airflow-env) bigdatacombains@bigdata-vm:~$ airflow dags details realtime_ecommerce_stream
/home/win10/airflow-env/lib/python3.10/site-packages/airflow/utils/dot_renderer.py:28 UserWarning: Could not import grap
hviz. Rendering graph to the graphical format will not be possible.
property_name | property_value
-----|-----
dag_id | realtime_ecommerce_stream
root_dag_id | None
is_paused | False
is_active | True
is_subdag | False
last_parsed_time | 2025-12-30T22:59:52.529053+00:00
last_pickled | None
last_expired | None
scheduler_lock | None
pickle_id | None
default_view | grid
fileloc | /home/win10/airflow/dags/realtime_stream_dag.py
file_token | Ii9ob2llL3dpbjEwL2FpcmZsb3cvZGFncy9yZWZsdGltZV9zdHJlYW1fZGFuLnB5Ig.4lu9bDKLRGzDpaSxGPZ
owners | bigdata-team
description | Symulacja real-time strumienia danych e-commerce (grudzień 2019 -> grudzień 2025)
schedule_interval | {'__type': 'CronExpression', 'value': '@daily'}
timetable_description | At 00:00
tags |
max_active_tasks | 16
max_active_runs | 1
has_task_concurrency_limits | False
has_import_errors | False
next_dagrun | 2025-12-30T00:00:00+00:00
next_dagrun_data_interval_start | 2025-12-30T00:00:00+00:00
next_dagrun_data_interval_end | 2025-12-31T00:00:00+00:00
next_dagrun_create_after | 2025-12-31T00:00:00+00:00
```

Rysunek 7: Statystyki działania DAG-u realtime_ecommerce_stream

Parametr start_date ustawiony na 26 grudnia 2025 definiuje pierwszy dzień, dla którego DAG może być uruchomiony. Jest to data wcześniejsza niż pierwszy dzień aktywnej symulacji (27 grudnia), co pozwala Airflow poprawnie zaplanować pierwsze wykonanie. Parametr retries=0 wskazuje że zadania które zakończą się niepowodzeniem nie będą automatycznie ponawiane, co jest właściwe dla symulacji długotrwałych, gdzie ponawianie mogłoby skutkować duplikacją danych w strumieniu.

DAG składa się z sześciu zadań wykonywanych sekwencyjnie, gdzie każde kolejne zadanie rozpoczyna się dopiero po pomyślnym zakończeniu poprzedniego. Zależności między zadaniami są wyrażone następująco: `check_date >> check_kafka >> cleanup_old_simulators >> ensure_topic >> run_simulator >> initial_monitor`, tworząc liniowy pipeline z jasno zdefiniowaną kolejnością wykonania. Operator `trigger_rule='all_done'` dla zadania monitorującego zapewnia, że monitoring jest wykonywany nawet jeśli wcześniejsze zadania zakończyły się niepowodzeniem, co umożliwia diagnostykę problemów. Rysunek 8 przedstawia widok Graph w interfejsie webowym Apache Airflow, wizualizując wszystkie sześć zadań DAG-u oraz ich wzajemne zależności, a także historię wykonania z kolorowymi wskaźnikami statusu (zielony dla sukcesu, czerwony dla niepowodzenia).



Rysunek 8: Graf w interfejsie graficznym Airflow reprezentujący DAG realtime_ecommerce_stream

7.2.2. Walidacja daty symulacji (check_date_eligibility)

Pierwsze zadanie DAG wykorzystuje PythonOperator do wykonania funkcji `check_date_eligibility()`, która weryfikuje, czy bieżąca data kwalifikuje się do symulacji. Funkcja sprawdza dwie reguły: (1) jeśli jest 27-31 grudnia, symulacja jest dozwolona dla odpowiadającego dnia grudnia 2019, (2) jeśli jest styczeń, symulacja jest dozwolona dla odpowiadających dni grudnia 2019 (dni 1-26 stycznia mapują się na dni 1-26 grudnia 2019).

Dla innych dat funkcja loguje ostrzeżenie i zwraca False, jednak nie przerywa wykonania DAG-u.

Logika walidacji daty jest zaimplementowana zarówno w samym symulatorze (metoda `load_events_for_today()`), jak i w tym zadaniu Airflow. Jeśli data nie kwalifikuje się do symulacji, symulator po prostu nie załaduje żadnych zdarzeń z CSV (ponieważ nie znajdzie odpowiadających dni w filtrze), a zadanie w Airflow zwróci komunikat informacyjny.

7.2.3. Weryfikacja infrastruktury Kafka (`check_kafka_health`)

Drugie zadanie wykorzystuje `BashOperator` do wykonania skryptu shell sprawdzającego, czy broker Kafka działa. Jeśli proces nie jest aktywny, skrypt automatycznie uruchamia Kafkę w trybie `daemon`, następnie czeka 10 sekund na pełną inicjalizację brokera. Po uruchomieniu lub potwierdzeniu działania Kafki, skrypt listuje dostępne topiki i weryfikuje obecność topiku `user-events`.

7.2.4. Czyszczenie nieaktualnych procesów (`cleanup_old_simulators`)

Trzecie zadanie identyfikuje i eliminuje "zombie" procesy symulatorów które mogły pozostać aktywne z poprzednich uruchomień. Skrypt znajduje wszystkie procesy uruchamiające `kafka_realtime_simulator.py`, następnie filtruje wyniki, by wykluczyć symulatory dla aktualnych dat (grudzień 27-31 oraz styczeń). Procesy, które nie pasują do tego kryterium są uznawane za zombie i są systematycznie eliminowane: najpierw przez standardowy `kill` (`SIGTERM`), następnie po 2 sekundach przez `kill -9` (`SIGKILL`), jeśli proces nadal działa.

Mechanizm `cleanup` zapobiega sytuacjom, gdzie wielokrotne wykonania DAG (np. po manualnym triggerze) mogłyby skutkować uruchomieniem wielu symulatorów publikujących duplikaty zdarzeń do Kafki. Dodatkowo czyszczenie starych plików PID usuwa pliki PID starsze niż 2 dni, zapobiegając zaśmieceniu systemu plików.

7.2.5. Konfiguracja topiku Kafki (`ensure_kafka_topic`)

Czwarte zadanie zapewnia, że topik `user-events` istnieje i jest poprawnie skonfigurowany. Topik jest tworzony z trzema partycjami dla równoległego przetwarzania, jednokrotną replikacją odpowiednią dla środowiska deweloperskiego oraz specyficznymi konfiguracjami: 7-dniową retencją, kompresją `gzip` oraz `segment.roll`'em co godzinę.

7.2.6. Uruchomienie symulatora (`run_realtime_simulator`)

Piąte zadanie odpowiada za faktyczne uruchomienie symulatora w tle z wykorzystaniem `nohup`. Skrypt implementuje inteligentną logikę wykrywania daty symulacji, rozróżniając między zaplanowanym uruchomieniem o północy a ręcznym wyzwoleniem w ciągu dnia. Dla planowanego uruchomienia używana jest data `execution_date + 1 day` (ponieważ w Airflow `execution_date` oznacza początek okresu, a symulacja dotyczy następnego dnia), podczas gdy dla ręcznego wyzwolenia używana jest bieżąca data systemowa.

Przed uruchomieniem nowego symulatora, skrypt sprawdza, czy istnieje plik PID dla danej daty oraz czy proces o tym identyfikatorze faktycznie działa. Jeśli aktywny symulator dla tej

daty już istnieje, skrypt loguje komunikat informacyjny i kończy wykonanie z kodem 0 (success), zapobiegając uruchomieniu duplikatu. Jest to dodatkowy mechanizm ochrony, spójny z parametrem `max_active_runs=1` na poziomie DAG-u, który chroni przed sytuacją, gdzie symulator został uruchomiony manualnie poza Airflow.

Komenda uruchomienia oparta o `nohup` realizuje kilka celów jednocześnie. Przede wszystkim zapewnia, że proces symulatora będzie kontynuował działanie nawet po zakończeniu sesji shell Airflow. Zmienna `$NEW_PID` przechwytuje identyfikator procesu uruchomionego w tle, który jest następnie zapisywany do pliku PID. Lokalizacja pliku PID oraz pliku log zawiera datę symulacji, co pozwala na rozróżnienie między równoległymi symulacjami różnych dat oraz ułatwia późniejszą diagnostykę przez wskazanie, która data była symulowana w którym pliku log.

7.2.7. Monitoring początkowego postępu (initial_monitor)

Szóste i ostatnie zadanie DAG-u wykonuje się 30 sekund po uruchomieniu symulatora, dając mu czas na inicjalizację i rozpoczęcie publikacji zdarzeń. Zadanie weryfikuje, czy proces symulatora nadal działa, wyświetla bieżące offsety partycji topiku `user-events` oraz prezentuje ostatnie 20 linii z logu symulatora. Te informacje pozwalają na szybką weryfikację, czy symulacja wystartowała poprawnie oraz czy zdarzenia faktycznie trafiają do Kafki.

Jeśli proces symulatora nie istnieje (co sugeruje natychmiastowe niepowodzenie przy starcie), zadanie monitorujące wyświetla ostatnie 50 linii logu i kończy się z kodem błędu 1, sygnalizując niepowodzenie w Airflow UI. Użycie `trigger_rule='all_done'` zapewnia, że monitoring wykonuje się nawet, jeśli wcześniejsze zadania miały problemy, co jest kluczowe dla diagnostyki. Wyświetlenie offsetów Kafki pozwala na weryfikację wzrostu liczby zdarzeń w czasie. Jeśli offsety nie rosną lub rosną asymetrycznie (jedna partycja ma znacząco więcej zdarzeń niż pozostałe), może to wskazywać na problemy z partycjonowaniem lub selektywnymi błędami publikacji dla pewnych kluczy użytkowników.

7.3. Archiwizacja strumienia do HDFS – Batch Layer

Archiwizacja danych strumieniowych do HDFS stanowi kluczowy element architektury Lambda, zapewniając długoterminowe przechowywanie surowych zdarzeń oraz umożliwiając analizy wsadowe. Proces archiwizacji konsumuje zdarzenia z Apache Kafka, transformuje je do efektywnego formatu kolumnowego Parquet z kompresją Snappy, oraz organizuje w hierarchicznej strukturze partycji umożliwiającej szybkie filtrowanie czasowe.

7.3.1. Architektura skryptu archiwizującego

Skrypt `kafka_to_hdfs_archiver.py` implementuje zoptymalizowany proces konsumpcji zdarzeń z Kafki oraz ich zapisu do HDFS, z naciskiem na efektywne zarządzanie pamięcią poprzez przetwarzanie wsadowe. Niniejsza implementacja dzieli przetwarzanie na batche po 250,000 zdarzeń każdy, co redukuje szczytowe zużycie pamięci RAM oraz eliminuje błędy `OutOfMemoryError`, które występowały we wcześniejszych wersjach systemu.

Inicjalizacja procesu archiwizacji wymaga podania dokładnej godziny do archiwizowania w formacie YYYY-MM-DD HH:MM:SS, na przykład 2025-12-30 10:00:00. Skrypt automatycznie oblicza przedział czasowy jednej godziny (od podanej godziny do godziny następnej) i konsumuje tylko te zdarzenia z Kafki, których stempel czasowy mieści się w tym przedziale. Wykorzystanie znacznika czasowego z Kafki zamiast timestampu logicznego ze zdarzenia zapewnia precyzyjne rozgraniczenie między godzinami, eliminując problemy ze zdarzeniami opóźnionymi lub przyszłymi, które mogłyby zaburzyć partycjonowanie.

Konsumer Kafki jest konfigurowany z unikalnym group ID opartym na zarchiwizowanej godzinie (hdfs-archiver-YYYYMMDDHH), co pozwala na równoległe uruchamianie wielu instancji archivera dla różnych godzin bez konfliktów offsetów. Parametr `auto_offset_reset='earliest'` zapewnia że konsumpcja rozpoczyna się od najstarszych dostępnych zdarzeń w topiku, co jest kluczowe dla operacji catch-up, gdy archiwizacja została uruchomiona z opóźnieniem.

7.3.2. Schemat danych i transformacje

Schemat danych zdefiniowany w funkcji `get_schema()` obejmuje wszystkie pola ze źródłowych zdarzeń Kafki oraz dodatkowe pola metadanych niezbędne dla diagnostyki. Podstawowe pola pochodzące z CSV (`event_time`, `event_type`, `product_id`, `category_id`, `category_code`, `brand`, `price`, `user_id`, `user_session`) są uzupełnione o trzy znaczniki czasowe dodane przez symulator (`original_time`, `mapped_time`, `processing_time`) oraz dwa pola metadanych Kafka (`kafka_timestamp`, `kafka_partition`). Rysunek 9 i Rysunek 10 prezentują pełny schemat DataFrame Spark dla zarchiwizowanych danych, pokazując 14 kolumn z odpowiednimi typami danych (`StringType`, `LongType`, `DoubleType`), oraz pięć reprezentatywnych wierszy ilustrujących faktyczną strukturę danych w formacie Parquet. Dane wyjściowe potwierdzają zachowanie wszystkich stempli czasowych oraz metadanych partycji Kafki, co umożliwia późniejszą analizę rozkładu zdarzeń oraz korelację między czasem logicznym a czasem przetworzenia.


```
scala> println("\n=== SAMPLE DATA (5 rows) ===")

=== SAMPLE DATA (5 rows) ===

scala> df.show(5, truncate=false)
+-----+-----+-----+-----+-----+-----+-----+-----+
|event_time|event_type|product_id|category_id|category_code|brand|price|
|user_id|user_session|original_time|mapped_time|processing_time|
|kafka_timestamp|kafka_partition|
+-----+-----+-----+-----+-----+-----+-----+-----+
|2019-12-30 10:00:00 UTC|view|38600028|2053013557225718275|apparel.shoes.sandals|gosh|6.3|
|514453622|4ad11043-e002-46f6-9b22-9d723d2d0cea|2019-12-30 10:00:00 UTC|2025-12-30T10:00:00.001158|1767088800001|2|
|2019-12-30 10:00:00 UTC|view|19300109|2232732109418529498|apparel.shirt|perilla|54.03|
|513211667|90daf1eb-366f-459b-a7a0-facb3cdb791f|2019-12-30 10:00:00 UTC|2025-12-30T10:00:00.002800|1767088800002|2|
|2019-12-30 10:00:00 UTC|view|3300492|2232732102414041665|computers.components.cooler|redmond|69.47|
|513858781|a416b5d5-0044-4c16-be08-13427a0d8403|2019-12-30 10:00:00 UTC|2025-12-30T10:00:00.003066|1767088800003|2|
|2019-12-30 10:00:00 UTC|purchase|1004708|2232732093077520756|construction.tools.light|huawei|148.86|
|512562069|252f5d0b-d766-4180-b1a6-57c31e95f9d5|2019-12-30 10:00:00 UTC|2025-12-30T10:00:00.003439|1767088800003|2|
|2019-12-30 10:00:00 UTC|cart|20100030|2232732110089618156|apparel.trousers|NULL|28.29|
|523174567|d2b2eee4-ec64-47f4-a298-2835bdf1c0dc|2019-12-30 10:00:00 UTC|2025-12-30T10:00:00.003825|1767088800003|2|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

scala> println("\n=== COUNT ===")

=== COUNT ===

scala> println(s"Total events: ${df.count()}")
Total events: 148484

scala> System.exit(0)
```

Rysunek 10: 5 przykładowych wierszy zapisanych w HDFS w katalogu
/raw/events/year=2025/month=12/day=30/hour=10

7.3.3. Przetwarzanie wsadowe i zapis do HDFS

Centralna pętla archiwizacji konsumuje zdarzenia z Kafki pojedynczo, filtrując je według stempla czasowego Kafki oraz agregując w liście pythonowej do momentu osiągnięcia rozmiaru batcha (250,000 zdarzeń). Po wypełnieniu batcha, lista jest konwertowana do Spark DataFrame z wykorzystaniem zdefiniowanego schematu, a następnie zapisywana do HDFS w formacie Parquet. Pierwszy batch jest zapisywany z trybem overwrite, aby usunąć ewentualne stare dane z poprzednich nieudanych uruchomień, podczas gdy kolejne batche używają trybu append aby nie nadpisywać już zapisanych danych. Po zapisie batcha, DataFrame jest usuwany, aby natychmiast zwolnić pamięć przed przetwarzaniem następnego batcha.

Ścieżka zapisu w HDFS jest konstruowana zgodnie z hierarchią partycjonowania: /raw/events/year=YYYY/month=MM/day=DD/hour=HH, gdzie wartości YYYY, MM, DD, HH pochodzą z zarchiwizowanej godziny. To partycjonowanie umożliwia automatyczne wykrywanie partycji przez Hive Metastore oraz efektywne przeszukiwanie partycji podczas zapytań analitycznych filtrujących po czasie. Rysunek 11 pokazuje strukturę katalogów dla grudnia 2025, z widocznymi trzema dniami (day=27, day=29, day=30), każdy reprezentujący jeden dzień symulacji. Rysunek 12 prezentuje szczegółową dekompozycję dnia 30 grudnia

na 18 partycji godzinowych (hour=00 do hour=17), pokazując regularność wykonywania DAG-a co godzinę.

```
(airflow-env) bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ hdfs dfs -ls /raw/events/year=2025/month=12
Found 3 items
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-28 00:03 /raw/events/year=2025/month=12/day=27
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 00:05 /raw/events/year=2025/month=12/day=29
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 18:09 /raw/events/year=2025/month=12/day=30
```

Rysunek 11: Struktura katalogu /raw/events/year=2025/month=12 w HDFS

```
(airflow-env) bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ hdfs dfs -ls /raw/events/year=2025/month=12/day=30
Found 18 items
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 01:05 /raw/events/year=2025/month=12/day=30/hour=00
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 02:06 /raw/events/year=2025/month=12/day=30/hour=01
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 03:06 /raw/events/year=2025/month=12/day=30/hour=02
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 04:06 /raw/events/year=2025/month=12/day=30/hour=03
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 05:06 /raw/events/year=2025/month=12/day=30/hour=04
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 06:14 /raw/events/year=2025/month=12/day=30/hour=05
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 07:08 /raw/events/year=2025/month=12/day=30/hour=06
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 08:07 /raw/events/year=2025/month=12/day=30/hour=07
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 09:08 /raw/events/year=2025/month=12/day=30/hour=08
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 10:08 /raw/events/year=2025/month=12/day=30/hour=09
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 11:08 /raw/events/year=2025/month=12/day=30/hour=10
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 12:08 /raw/events/year=2025/month=12/day=30/hour=11
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 13:36 /raw/events/year=2025/month=12/day=30/hour=12
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 14:09 /raw/events/year=2025/month=12/day=30/hour=13
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 15:08 /raw/events/year=2025/month=12/day=30/hour=14
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 16:09 /raw/events/year=2025/month=12/day=30/hour=15
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 17:08 /raw/events/year=2025/month=12/day=30/hour=16
drwxr-xr-x - bigdatacombains supergroup          0 2025-12-30 18:09 /raw/events/year=2025/month=12/day=30/hour=17
```

Rysunek 12: Struktura katalogu /raw/events/year=2025/month=12/day=30 w HDFS

Rysunek 13 pokazuje zawartość pojedynczej partycji godzinowej, składającą się z dwóch plików Parquet o rozmiarach 2.7 MB i 2.6 MB, oraz pliku `_SUCCESS` sygnalizującego pomyślne zakończenie operacji zapisu. Obecność dwóch plików Parquet wynika z konfiguracji Spark'a obejmującej dwa executory, gdzie każdy executor zapisuje swój fragment danych. Sufiksy `.snappy.parquet` potwierdzają zastosowanie kompresji Snappy. Rysunek 14 prezentuje statystyki wykorzystania przestrzeni HDFS: całkowity rozmiar archiwum wynosi 224.9 MB, z podziałem na 64.7 MB dla 27 grudnia, 88.8 MB dla 29 grudnia oraz 71.5 MB dla 30 grudnia, odzwierciedlając różną liczbę zarchiwizowanych godzin oraz zmienną intensywność strumienia zdarzeń.

```
(airflow-env) bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ hdfs dfs -ls -h /raw/events/year=2025/month=12/day=30/hour=10
Found 3 items
-rw-r--r-- 1 bigdatacombains supergroup          0 2025-12-30 11:08 /raw/events/year=2025/month=12/day=30/hour=10/_SUCCESS
-rw-r--r-- 1 bigdatacombains supergroup      2.7 M 2025-12-30 11:08 /raw/events/year=2025/month=12/day=30/hour=10/part-00000-cccf2981-62a4-49d9-9e08-01b1c20fdd42-c000.snappy.parquet
-rw-r--r-- 1 bigdatacombains supergroup      2.6 M 2025-12-30 11:08 /raw/events/year=2025/month=12/day=30/hour=10/part-00001-cccf2981-62a4-49d9-9e08-01b1c20fdd42-c000.snappy.parquet
```

Rysunek 13: Zawartość katalogu /raw/events/year=2025/month=12/day=30/hour=10 w HDFS

```
(airflow-env) bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ hdfs dfs -du -h -s /raw/events
224.9 M 224.9 M /raw/events
(airflow-env) bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ hdfs dfs -du -h /raw/events/year=2025/month=12/ | sort
64.7 M 64.7 M /raw/events/year=2025/month=12/day=27
71.5 M 71.5 M /raw/events/year=2025/month=12/day=30
88.8 M 88.8 M /raw/events/year=2025/month=12/day=29
```

Rysunek 14: Statystyki zawartości katalogu /raw/events/year=2025/month=12 w HDFS

Logi dostarczają szczegółowych metryk postępu: co 10,000 zdarzeń logowany jest komunikat o liczbie zebranych zdarzeń, przy zapisie każdego batcha logowana jest jego liczba porządkowa oraz rozmiar, a po zakończeniu całej operacji logowana jest sumaryczna liczba zdarzeń oraz liczba batchy. Te informacje umożliwiają szybką identyfikację anomalii, takich jak nieoczekiwanie mała liczba zdarzeń lub nadmiernie długi czas przetwarzania.

7.4. Orkiestracja archiwizacji przez Apache Airflow

7.4.1. Struktura DAG kafka_to_hdfs_archiver

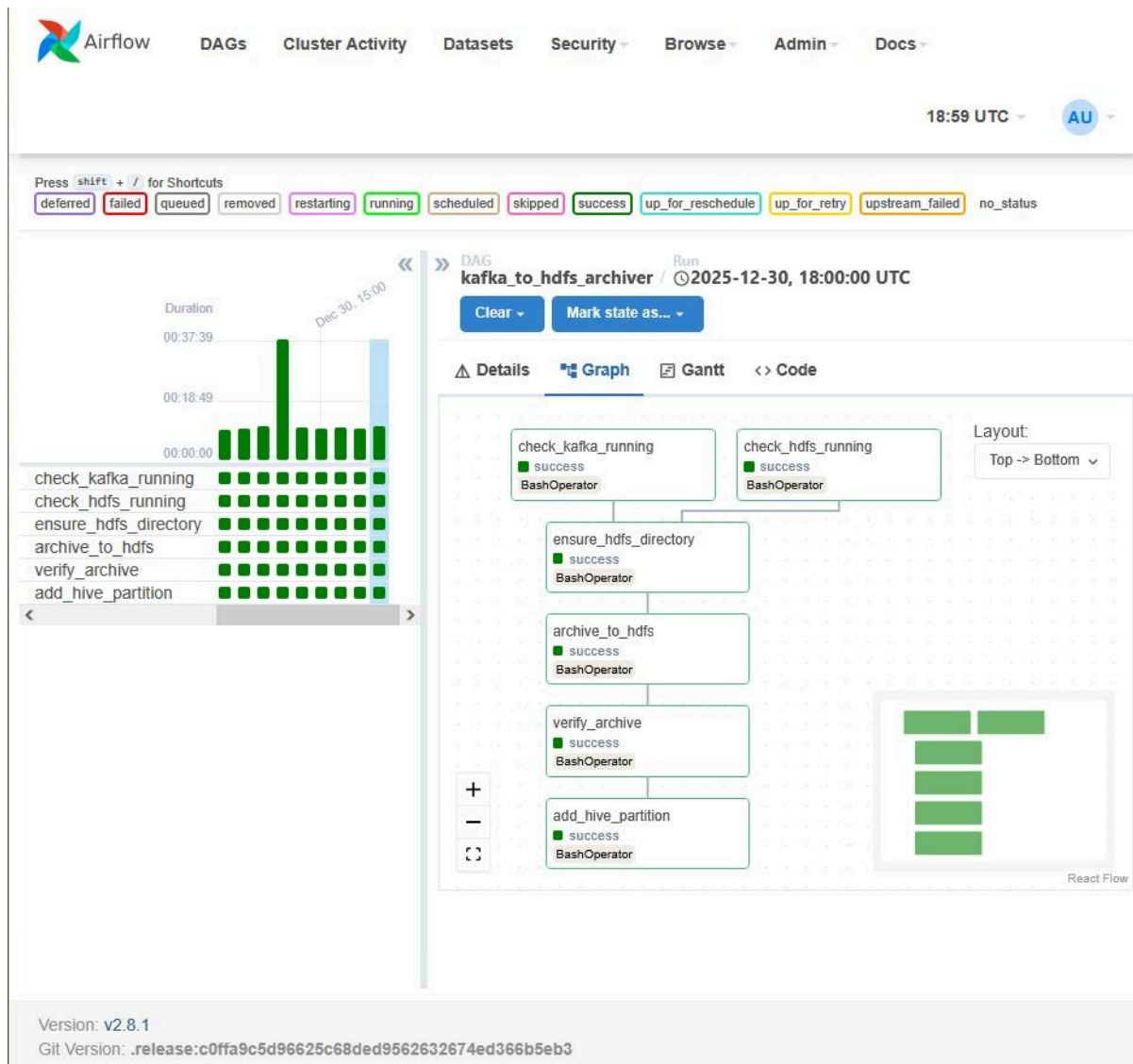
DAG o identyfikatorze kafka_to_hdfs_archiver realizuje periodyczną archiwizację strumienia Kafka do HDFS zgodnie z harmonogramem @hourly, wykonując się co godzinę o pełnej godzinie (XX:00). Kluczową cechą konfiguracji jest catchup=True, które włącza mechanizm historycznego odtwarzania wykonania dla godzin, które minęły między datą start_date a bieżącą godziną. Dzięki temu, jeśli DAG zostanie aktywowany po raz pierwszy 30 grudnia o godzinie 12:00, Airflow automatycznie zaplanuje wykonania dla wszystkich godzin od start_date (27 grudnia 01:00) do 12:00 włącznie, zapewniając kompletność archiwum bez ręcznej interwencji.

Rysunek 15 prezentuje szczegółowe właściwości DAG-u. Kluczowe parametry obejmują: schedule_interval: @hourly potwierdzający godzinową częstotliwość, max_active_runs: 1 zapewniający że tylko jedna instancja DAG wykonuje się w danym momencie (zapobiegając konfliktom przy dostępie do tych samych partycji HDFS), is_paused: False wskazujący że DAG jest aktywny, oraz next_dagrun: 2025-12-30T18:00:00 pokazujący, że następne zaplanowane wykonanie nastąpi o 18:00 dla godziny 18:00-19:00.


```
(airflow-env) bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ airflow dags details kafka_to_hdfs_archiver
/home/win10/airflow-env/lib/python3.10/site-packages/airflow/utils/dot_renderer.py:28 UserWarning: Could not
import graphviz. Rendering graph to the graphical format will not be possible.
property_name | property_value
=====
dag_id | kafka_to_hdfs_archiver
root_dag_id | None
is_paused | False
is_active | True
is_subdag | False
last_parsed_time | 2025-12-30T18:54:17.885484+00:00
last_pickled | None
last_expired | None
scheduler_lock | None
pickle_id | None
default_view | grid
fileloc | /home/win10/airflow/dags/kafka_to_hdfs_dag.py
file_token | Ii9ob2llL3dpbjEwL2FpcmZsb3cvZGFncy9rYWZrYV90b19oZGZzX2RhZy5weSI.q3RTFEMPqL
NQeIU9hQVklvm3TeU
owners | bigdata-team
description | Archiwizuje eventy z Kafki do HDFS co godzinę
schedule_interval | {'__type': 'CronExpression', 'value': '@hourly'}
timetable_description | Every hour
tags | {'name': 'archiving'}, {'name': 'hdfs'}, {'name': 'streaming'}, {'name':
'kafka'}
max_active_tasks | 16
max_active_runs | 1
has_task_concurrency_limits | False
has_import_errors | False
next_dagrun | 2025-12-30T18:00:00+00:00
next_dagrun_data_interval_start | 2025-12-30T18:00:00+00:00
next_dagrun_data_interval_end | 2025-12-30T19:00:00+00:00
next_dagrun_create_after | 2025-12-30T19:00:00+00:00
```

Rysunek 15: Statystyki działania DAG-u kafka_to_hdfs_archiver

DAG składa się z sześciu zadań wykonywanych w sekwencji z rozgałęzieniem na początku: [check_kafka_running, check_hdfs_running] wykonują się równolegle jako walidacja infrastruktury, następnie ensure_hdfs_directory tworzy katalog bazowy jeśli nie istnieje, po czym archive_to_hdfs wykonuje faktyczną archiwizację, verify_archive weryfikuje poprawność zapisu, a na końcu add_hive_partition rejestruje nową partycję w Hive Metastore. Rysunek 16 wizualizuje tę strukturę w interfejsie webowym Airflow, pokazując sześć bloków zadań oraz strzałki reprezentujące zależności między nimi, z kolorowym kodowaniem statusu wykonania dla historycznych uruchomień.



Rysunek 16: Graf w interfejsie graficznym Airflow reprezentujący DAG kafka_to_hdfs_archiver

7.4.2. Walidacja infrastruktury (check_kafka, check_hdfs)

Zadanie check_kafka_running zajmuje się wykryciem procesu brokera Kafki, a w przypadku jego braku uruchamia go z 10-sekundowym oczekiwaniem na pełną inicjalizację. Analogicznie, check_hdfs_running wykrywa NameNode i w razie potrzeby uruchamia HDFS z 15-sekundowym oczekiwaniem. Te zadania są krytyczne dla niezawodności archiwizacji w środowisku, gdzie serwisy mogą być czasowo niedostępne (np. po restarcie VM lub po planowanej konserwacji).

7.4.3. Przygotowanie środowiska i wykonanie archiwizacji

Zadanie `ensure_hdfs_directory` tworzy katalog bazowy, pod warunkiem, że jeszcze nie istnieje, eliminując błędy przy wielokrotnym wykonaniu.

Zadanie `archive_to_hdfs` wykonuje `spark-submit` ze skrypcem `kafka_to_hdfs_archiver.py`. Parametr `--master local[2]` uruchamia Spark w trybie lokalnym z dwoma executorami, co jest optymalne dla pojedynczej maszyny oraz pozwala na równoległy zapis dwóch plików do HDFS. Konfiguracja `--driver-memory 2g` alokuje 2 GB RAM dla drivera Spark, co jest wystarczające przy przetwarzaniu wsadowym po 250k zdarzeń.

7.4.4. Weryfikacja i rejestracja partycji

Zadanie `verify_archive` wykonuje weryfikację, że dane faktycznie zostały zapisane do HDFS poprzez sprawdzenie istnienia katalogu partycji oraz zliczenie plików Parquet w nim. Zadanie wyświetla także całkowity rozmiar partycji, co dostarcza cennych metryk dla monitorowania wzrostu archiwum. Jeśli katalog partycji nie istnieje, task kończy się niepowodzeniem (`exit 1`), sygnalizując problem wymagający interwencji. Jeśli katalog istnieje, ale nie zawiera plików Parquet, task kończy się sukcesem (`exit 0`) z komunikatem informacyjnym, ponieważ brak danych może być poprawną sytuacją (np. żadne zdarzenia nie wystąpiły w danej godzinie).

Zadanie `add_hive_partition` wykonuje komendę `spark-sql` z poleceniem `ALTER TABLE user_events ADD IF NOT EXISTS PARTITION`. Dynamiczne wartości partycji (`year`, `month`, `day`, `hour`) są podstawiane przez Airflow templates, a `LOCATION` wskazuje na odpowiedni katalog w HDFS z właściwym formatowaniem zer wiodących. Jeśli partycja już istnieje, polecenie kończy się sukcesem bez duplikacji metadanych.

Rejestracja partycji w Hive Metastore jest kluczowa dla późniejszych zapytań SQL przez Spark oraz Hive, ponieważ bez metadanych partycji system nie wie, że dane dla danej godziny istnieją, nawet jeśli pliki Parquet są fizycznie obecne w HDFS. Błędy podczas rejestracji partycji (np. problem z połączeniem do Metastore) są tolerowane, ponieważ najważniejsze jest, że dane faktycznie są zapisane w HDFS – partycję można zarejestrować manualnie później, podczas gdy brak danych w HDFS jest nieodwracalny.

7.4.5. Historia wykonania i metryki

Rysunek 17 prezentuje historię ośmiu ostatnich wykonania DAG, pokazując `execution_date`, `run_id`, `state`, `start_date` oraz `end_date`. Analiza stempli czasowych pokazuje regularną periodyczność: każdy run rozpoczyna się o pełnej godzinie następnej po `execution_date`, co jest zgodne z semantyką Airflow, gdzie `execution_date` oznacza początek okresu danych, a faktyczne wykonanie następuje po zakończeniu tego okresu. Czas trwania każdego runu wynosi około 9-10 minut, co obejmuje czas na konsumpcję zdarzeń z Kafki, przetwarzanie wsadowe oraz zapis do HDFS.

```
(airflow-env) bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ airflow dags list-runs -d kafka_to_hdfs_archiver
```

/home/win10/airflow-env/lib/python3.10/site-packages/airflow/utils/dot_renderer.py:28 UserWarning: Could not import graphviz. Rendering graph to the graphical format will not be possible.

dag_id	run_id	state	execution_date	start_date	end_date
kafka_to_hdfs_archiver	scheduled_2025-12-30T17:00:00+00:00	success	2025-12-30T17:00:00+00:00	2025-12-30T18:00:01.098917+00:00	2025-12-30T18:10:26.917283+00:00
kafka_to_hdfs_archiver	scheduled_2025-12-30T16:00:00+00:00	success	2025-12-30T16:00:00+00:00	2025-12-30T17:00:01.373796+00:00	2025-12-30T17:09:50.896585+00:00
kafka_to_hdfs_archiver	scheduled_2025-12-30T15:00:00+00:00	success	2025-12-30T15:00:00+00:00	2025-12-30T16:00:02.029419+00:00	2025-12-30T16:10:01.874989+00:00
kafka_to_hdfs_archiver	scheduled_2025-12-30T14:00:00+00:00	success	2025-12-30T14:00:00+00:00	2025-12-30T15:00:01.069310+00:00	2025-12-30T15:09:51.465300+00:00
kafka_to_hdfs_archiver	scheduled_2025-12-30T13:00:00+00:00	success	2025-12-30T13:00:00+00:00	2025-12-30T14:00:01.527010+00:00	2025-12-30T14:09:56.879599+00:00
kafka_to_hdfs_archiver	scheduled_2025-12-30T12:00:00+00:00	success	2025-12-30T12:00:00+00:00	2025-12-30T13:00:02.113428+00:00	2025-12-30T13:37:41.545240+00:00
kafka_to_hdfs_archiver	scheduled_2025-12-30T11:00:00+00:00	success	2025-12-30T11:00:00+00:00	2025-12-30T12:00:01.430590+00:00	2025-12-30T12:10:24.361028+00:00
kafka_to_hdfs_archiver	scheduled_2025-12-30T10:00:00+00:00	success	2025-12-30T10:00:00+00:00	2025-12-30T11:00:01.449859+00:00	2025-12-30T11:09:40.214945+00:00
kafka_to_hdfs_archiver	scheduled_2025-12-30T09:00:00+00:00	success	2025-12-30T09:00:00+00:00	2025-12-30T10:00:01.938378+00:00	2025-12-30T10:09:23.533218+00:00

Rysunek 17: Przykładowe wykonania DAG-u kafka_to_hdfs_archiver

Logi poszczególnych tasków dostępne w Airflow UI zawierają szczegółowe metryki, takie jak liczba zarchiwizowanych zdarzeń, liczba batchy, rozmiar plików Parquet oraz czas trwania każdego etapu.

Rysunek 9 i Rysunek 10 dodatkowo pokazują końcowy rezultat całego pipeline'u archiwizacji: dane w formacie Parquet z pełnym schematem 14 kolumn, poprawnie zmapowanymi typami danych oraz zachowanymi wszystkimi metadanymi pochodzącymi z różnych etapów przetwarzania. Liczba 148,484 zdarzeń dla godziny 10:00 dnia 30 grudnia odpowiada typowej przepustowości symulatora (~ 41 zdarzeń/sekundę * 3600 sekund $\approx 147,600$ zdarzeń), potwierdzając kompletność archiwizacji bez utraty danych.

7.5. Analiza danych strumieniowych w czasie rzeczywistym – Spark Streaming

Analityka w czasie rzeczywistym realizowana jest przez dedykowany skrypt PySpark realtime_analytics.py, który konsumuje zdarzenia z Kafki, przetwarza je w pięciominutowych oknach czasowych z minutowym przesunięciem oraz zapisuje wyniki do HBase dla natychmiastowego dostępu przez aplikacje klienckie. System implementuje pięć równoległych strumieni analitycznych, każdy obliczający inne metryki biznesowe z wykorzystaniem Spark Structured Streaming API.

7.5.1. Architektura procesu Spark Streaming

Proces Spark Streaming działa jako long-running job uruchomiony przez spark-submit w trybie local[4], co oznacza wykorzystanie czterech lokalnych executorów dla

przetwarzania równoległego. Rysunek 18 pokazuje dwa procesy związane z jobem: główny proces Java Spark zużywający 103% CPU i 32.8% pamięci (~5GB z 14GB), oraz proces Python driver zużywający 1.0% CPU i 0.6% pamięci (~100MB). Intensywne zużycie CPU przez proces Java (103% oznacza wykorzystanie ponad jednego rdzenia z dostępnych czterech) potwierdza aktywne przetwarzanie strumieniowego, podczas gdy niskie zużycie przez driver Python wskazuje że większość obliczeń odbywa się w warstwach Spark wykonywanych przez JVM.

```
(airflow-env) bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ ps aux | grep realtime_analytics | grep -v grep
bigdata+ 2605472  103 32.9 8305564 5054692 ?        Ssl  13:22 417:49 /usr/lib/jvm/java-8-openjdk-amd64/bin/java -cp /opt/spark/conf:/opt/spark/jars/*:/opt/hadoop/etc/hadoop:/opt/hadoop/share/hadoop/common/lib/*:/opt/hadoop/share/hadoop/common/*:/opt/hadoop/share/hadoop/hdfs/*:/opt/hadoop/share/hadoop/hdfs/lib/*:/opt/hadoop/share/hadoop/hdfs/*:/opt/hadoop/share/hadoop/mapreduce/*:/opt/hadoop/share/hadoop/yarn/*:/opt/hadoop/share/hadoop/yarn/lib/*:/opt/hadoop/share/hadoop/yarn/* -Xmx4g -XX:+IgnoreUnrecognizedVMOptions --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.base/java.net=ALL-UNNAMED --add-opens=java.base/java.nio=ALL-UNNAMED --add-opens=java.base/java.util=ALL-UNNAMED --add-opens=java.base/java.util.concurrent.atomic=ALL-UNNAMED --add-opens=java.base/jdk.internal.ref=ALL-UNNAMED --add-opens=java.base/sun.nio.ch=ALL-UNNAMED --add-opens=java.base/sun.nio.cs=ALL-UNNAMED --add-opens=java.base/sun.security.action=ALL-UNNAMED --add-opens=java.base/sun.util.calendar=ALL-UNNAMED --add-opens=java.security.jgss/sun.security.krb5=ALL-UNNAMED -Djdk.reflect.useDirectMethodHandle=false org.apache.spark.deploy.SparkSubmit --master local[4] --conf spark.driver.memory=4g --executor-memory 2g --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3 realtime_analytics.py
bigdata+ 2605703  1.0 0.6 896708 103676 ?        Ssl  13:22  4:20 python3 /home/win10/scripts/realtime_analytics.py
```

Rysunek 18: Procesy stworzone w ramach realtime_analytics

Rysunek 19 prezentuje szczegółowe metryki procesu Java Spark, ujawniając elapsed time 07:00:41 (ponad 7 godzin nieprzerwanie działającego streaming job'a), co demonstruje długoterminową stabilność systemu. Konfiguracja pamięci -Xmx4g w komendzie uruchomienia alokuje 4 GB dla drivera Spark, co wraz z pamięcią executorów daje łączne ~5GB, zgodnie z obserwowanym zużyciem. Parent Process ID (PPID) równy 1 wskazuje, że proces został "adoptowany" przez init/systemd po zakończeniu oryginalnego parent process (prawdopodobnie nohup), co jest normalnym zachowaniem dla procesów uruchomionych w tle i potwierdza właściwą izolację od terminala.

```
bigdatacombains@bigdata-vm:~$ ps -p $PID -o pid,ppid,%mem,%cpu,etime,cmd
PID    PPID %MEM %CPU  ELAPSED CMD
2605472 1 32.9 103 07:00:41 /usr/lib/jvm/java-8-openjdk-amd64/bin/java -cp /opt/spark/conf:/opt/spark/jars/*
```

Rysunek 19: Statystyki procesu realtime_analytics

Spark Session jest konfigurowany z opcjami zoptymalizowanymi dla przetwarzania strumieniowego: spark.streaming.stopGracefullyOnShutdown=true zapewnia, że zatrzymanie streaming job spowoduje zamknięcie zasobów, spark.sql.streaming.checkpointLocation wskazuje katalog /home/win10/spark-checkpoints/realtime dla checkpointów umożliwiających powrót do działania po awarii, spark.driver.memory=4g alokuje 4GB dla drivera (większość logiki streaming job wykonuje się tu), spark.executor.memory=2g alokuje 2GB dla każdego executora oraz spark.sql.shuffle.partitions=8 ustawia liczbę partycji shuffle na 8 dla operacji agregacji.

7.5.2. Konsumpcja zdarzeń z Kafki

Streaming DataFrame jest tworzony z subskrypcją na topik user-events oraz opcją `startingOffsets="latest"`, która powoduje, że nowo uruchomiony streaming job rozpoczyna konsumpcję od najnowszych dostępnych zdarzeń zamiast od początku topiku (co jest właściwe dla realtime analytics, gdzie historyczne dane są mniej istotne). Opcja `failOnDataLoss=false` toleruje sytuacje, gdzie dane w Kafce zostały usunięte przez politykę retencji, zanim streaming job zdążył je skonsumentować, co jest akceptowalne w kontekście metrics, które są agregowane w oknach czasowych i tracą wartość po pewnym czasie.

Każdy rekord z Kafki zawiera pole `value` z JSON-a oraz stempel czasowy wskazujący czas pojawienia się tego zdarzenia w Kafce. Pole `value` jest deserializowane, następnie parsowane z JSON-a do struktury zgodnej ze schematem zdefiniowanym w `get_event_schema()`. Struktura obejmuje 12 pól: podstawowe pola biznesowe (`event_time`, `event_type`, `product_id`, `category_id`, `category_code`, `brand`, `price`, `user_id`, `user_session`) oraz pola metadanych dodane przez symulator (`original_time`, `mapped_time`, `processing_time`). Timestamp `event_time` jest następnie konwertowany do typu `timestamp` Spark (`to_timestamp`) aby umożliwić operacje na oknach czasowych.

7.5.3. Pięć równoległych analiz strumieniowych

Analiza 1: Aktywni użytkownicy

Pierwsza analiza oblicza liczbę unikalnych użytkowników aktywnych w danym oknie czasowym przez agregację `approx_count_distinct("user_id")`, która wykorzystuje algorytm HyperLogLog dla efektywnego oszacowania liczby elementów bez konieczności przechowywania wszystkich unikalnych wartości w pamięci. 10-minutowy watermark definiuje, jak długo system czeka na spóźnione zdarzenia przed finalizacją okna – zdarzenia spóźnione o więcej niż 10 minut są odrzucane. Okno 5-minutowe z 1-minutowym przesunięciem tworzy sytuację, w której każdy nowy wynik jest generowany co minutę, a każde zdarzenie może być częścią 5 różnych okien.

Analiza 2: Top 10 produktów (wyświetlenia)

Druga analiza identyfikuje 10 najpopularniejszych produktów pod względem liczby wyświetleń w oknie czasowym. Filtr ogranicza dane do zdarzeń typu "view", następnie agregacja grupuje je po oknie czasowym, id produktu, marce oraz kodzie kategorii, obliczając liczbę wyświetleń oraz średnią cenę. Funkcja `foreachBatch` w strumieniu wyjściowym wybiera top 10 produktów przed zapisaniem do HBase, co ogranicza liczbę rekordów zapisywanych dla każdego okna do maksymalnie 10.

Analiza 3: Metryki zakupów

Trzecia analiza agreguje wszystkie zdarzenia typu "purchase" w oknie czasowym, obliczając cztery kluczowe metryki e-commerce: liczbę zakupów, łączny przychód, średnią wartość koszyka oraz liczbę unikalnych użytkowników dokonujących zakupów. Te metryki dostarczają umożliwiając natychmiastową detekcję anomalii, takich jak nagły spadek liczby zakupów czy drastyczna zmiana średniej wartości koszyka.

Analiza 4: Rozkład typów zdarzeń

Czwarta analiza liczy wystąpienia każdego typu zdarzenia (view, cart, purchase) w oknie czasowym, dostarczając przegląd aktywności użytkowników. Agregacja grupuje po oknie czasowym oraz typie zdarzenia, obliczając licznosc każdego typu. Ten rozkład pozwala na obliczenie metryk, takich jak współczynnik view-to-cart oraz współczynnik konwersji cart-to-purchase, które są kluczowe dla optymalizacji ścieżki zakupowej.

Analiza 5: Top 10 marek według aktywności

Piąta analiza identyfikuje najpopularniejsze marki według liczby zdarzeń każdego typu. Filtr `filter(col("brand").isNull())` eliminuje zdarzenia bez przypisanej marki, następnie agreguje i grupuje je po oknie czasowym, marce oraz typie zdarzenia, obliczając liczbę zdarzeń oraz całkowitą wartość transakcji. Podział według typu zdarzenia pozwala na analizę, czy dana marka generuje głównie wyświetlenia, jest dodawana do koszyka, czy też kupowana.

7.5.4. Zapis wyników do HBase

Każda analiza wykorzystuje wyrażenie `lambda foreachBatch`, które jest wywoływane dla każdego `micro-batch'a`. Rysunek 20 pokazuje typowy output logowania, gdzie każda epoka generuje 5 wywołań funkcji zapisu, po jednym dla każdej analizy. Logi zawierają potwierdzenia zapisu w formacie "[EPOCH XXXX] Zapisano YY rekordów TYP_ANALIZY" do HBase, gdzie liczba rekordów odpowiada naturze analizy (np. 5 dla `active_users`, 10 dla `top_products_views`, 10 dla `top_brands`). Ostrzeżenia "[EPOCH XXXX] Brak danych purchases - pomijam" występują dla okien czasowych, gdzie nie było żadnych zakupów.

```

bigdatacombains@bigdata-vm:~$ tail -50 /home/win10/airflow/logs/realtime_analytics.log
2025-12-30 20:24:34,993 - INFO - [EPOCH 5311] Zapisano 5 rekordów purchases do HBase
2025-12-30 20:24:34,994 - INFO - [EPOCH 6295] Zapisuję top_brands do HBase...
2025-12-30 20:24:35,146 - INFO - Received command c on object id p0
2025-12-30 20:24:35,153 - INFO - [EPOCH 5414] Zapisuję active_users do HBase...
2025-12-30 20:24:35,232 - INFO - Received command c on object id p1
2025-12-30 20:24:35,238 - INFO - [EPOCH 6245] Zapisuję top_products_views do HBase...
2025-12-30 20:24:35,592 - INFO - Received command c on object id p2
2025-12-30 20:24:35,612 - INFO - [EPOCH 5312] Zapisuję purchases do HBase...
2025-12-30 20:24:36,743 - INFO - [EPOCH 6130] Zapisano 10 rekordów event_distribution do HBase
2025-12-30 20:24:37,317 - INFO - Received command c on object id p3
2025-12-30 20:24:37,324 - INFO - [EPOCH 6131] Zapisuję event_distribution do HBase...
2025-12-30 20:24:37,886 - INFO - [EPOCH 6295] Zapisano 10 rekordów top_brands do HBase
2025-12-30 20:24:38,066 - INFO - [EPOCH 5414] Zapisano 5 rekordów active_users do HBase
2025-12-30 20:24:38,067 - INFO - Received command c on object id p4
2025-12-30 20:24:38,074 - INFO - [EPOCH 6296] Zapisuję top_brands do HBase...
2025-12-30 20:24:38,212 - INFO - Received command c on object id p0
2025-12-30 20:24:38,218 - INFO - [EPOCH 5415] Zapisuję active_users do HBase...
2025-12-30 20:24:38,886 - INFO - [EPOCH 6245] Zapisano 10 rekordów top_products_views do HBase
2025-12-30 20:24:39,131 - INFO - Received command c on object id p1
2025-12-30 20:24:39,137 - INFO - [EPOCH 6246] Zapisuję top_products_views do HBase...
2025-12-30 20:24:40,040 - WARNING - [EPOCH 5312] Brak danych purchases - pomijam
2025-12-30 20:24:40,521 - INFO - [EPOCH 6131] Zapisano 15 rekordów event_distribution do HBase
2025-12-30 20:24:40,637 - INFO - Received command c on object id p2
2025-12-30 20:24:40,644 - INFO - [EPOCH 5313] Zapisuję purchases do HBase...
2025-12-30 20:24:40,948 - INFO - [EPOCH 6296] Zapisano 10 rekordów top_brands do HBase
2025-12-30 20:24:41,440 - INFO - [EPOCH 5415] Zapisano 5 rekordów active_users do HBase
2025-12-30 20:24:41,493 - INFO - Received command c on object id p3
2025-12-30 20:24:41,499 - INFO - [EPOCH 6132] Zapisuję event_distribution do HBase...
2025-12-30 20:24:41,539 - INFO - Received command c on object id p4
2025-12-30 20:24:41,546 - INFO - [EPOCH 6297] Zapisuję top_brands do HBase...
2025-12-30 20:24:42,013 - INFO - Received command c on object id p0
2025-12-30 20:24:42,022 - INFO - [EPOCH 5416] Zapisuję active_users do HBase...
2025-12-30 20:24:42,134 - INFO - [EPOCH 6246] Zapisano 10 rekordów top_products_views do HBase
2025-12-30 20:24:42,324 - INFO - Received command c on object id p1
2025-12-30 20:24:42,330 - INFO - [EPOCH 6247] Zapisuję top_products_views do HBase...
2025-12-30 20:24:43,994 - INFO - [EPOCH 5313] Zapisano 5 rekordów purchases do HBase
2025-12-30 20:24:44,151 - INFO - [EPOCH 6132] Zapisano 15 rekordów event_distribution do HBase
2025-12-30 20:24:44,298 - INFO - Received command c on object id p3
2025-12-30 20:24:44,304 - INFO - [EPOCH 6133] Zapisuję event_distribution do HBase...
2025-12-30 20:24:44,554 - INFO - Received command c on object id p2
2025-12-30 20:24:44,562 - INFO - [EPOCH 5314] Zapisuję purchases do HBase...
2025-12-30 20:24:44,685 - INFO - [EPOCH 5416] Zapisano 5 rekordów active_users do HBase
2025-12-30 20:24:44,740 - INFO - [EPOCH 6297] Zapisano 10 rekordów top_brands do HBase
2025-12-30 20:24:45,162 - INFO - [EPOCH 6247] Zapisano 10 rekordów top_products_views do HBase
2025-12-30 20:24:45,237 - INFO - Received command c on object id p0
2025-12-30 20:24:45,242 - INFO - [EPOCH 5417] Zapisuję active_users do HBase...
2025-12-30 20:24:45,334 - INFO - Received command c on object id p4
2025-12-30 20:24:45,340 - INFO - [EPOCH 6298] Zapisuję top_brands do HBase...
2025-12-30 20:24:45,765 - INFO - Received command c on object id p1
2025-12-30 20:24:45,771 - INFO - [EPOCH 6248] Zapisuję top_products_views do HBase...

```

Rysunek 20: Logi procesu realtime_analytics

Połączenie z HBase realizowane jest przez bibliotekę happybase komunikującą się z HBase Thrift Server na porcie 9090. Każde wywołanie funkcji zapisu tworzy nowe połączenie, otwiera tabelę realtime_stats, zapisuje rekordy, oraz zamyka połączenie. Row key w HBase jest konstruowany jako {timestamp}{typ_analizy}_{szczegóły}. Ta hierarchia klucza umożliwia efektywne skanowanie wszystkich metryk dla danego okna czasowego przez skan wykorzystujący STARTROW i STOPROW.

Rysunek 21 prezentuje strukturę tabeli HBase z dwiema rodzinami kolumn: metrics z 10 wersjami historycznymi dla przechowywania ewolucji wartości w czasie oraz metadata z 3 wersjami dla metadanych, takich jak stempel czasowy okna oraz stempel czasowy zapisu. Kompresja i pamięć podręczna są wyłączone.

```

bigdatacombains@bigdata-vm:~$ echo "describe 'realtime_stats'" | hbase shell
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/hbase/lib/client-facing-thirdparty/log4j-slf4j-impl-2.17.2.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: http://hbase.apache.org/2.0/book.html#shell
Version 2.6.4-hadoop3, r67f414cfed130bf6325570463ed36ea7d45aebc5, Mon Nov 3 19:30:14 PST 2025
Took 0.0031 seconds
hbase:001:0> describe 'realtime_stats'
Table realtime_stats is ENABLED
realtime_stats, {TABLE_ATTRIBUTES => {METADATA => {'hbase.store.file-tracker.impl' => 'DEFAULT'}}}
COLUMN FAMILIES DESCRIPTION
(NAME => 'metadata', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '3', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)')
(NAME => 'metrics', INDEX_BLOCK_ENCODING => 'NONE', VERSIONS => '10', KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING => 'NONE', TTL => 'FOREVER', MIN_VERSIONS => '0', REPLICATION_SCOPE => '0', BLOOMFILTER => 'NONE', IN_MEMORY => 'false', COMPRESSION => 'NONE', BLOCKCACHE => 'false', BLOCKSIZE => '65536 B (64KB)')
2 row(s)
Quota is disabled
Took 1.7202 seconds
hbase:002:0>

```

Rysunek 21: Opis tabeli realtime_stats

Rysunek 22 pokazuje wynik komendy count 'realtime_stats' który zwrócił 67,114 wierszy, co reprezentuje skumulowane wyniki analiz przez około 7 godzin działania streaming job'a (od około 02:00 do 21:00 dnia 30 grudnia).

```

67114 row(s)
Took 3.4771 seconds
=> 67114
hbase:002:0>

```

Rysunek 22: Liczba wierszy zgromadzonych w HBase (stan na 30 grudnia, godzina 21:00)

7.5.5. Przykładowe wyniki analiz

Rysunek 23 prezentuje zagregowane wyniki pięciu równoległych analiz strumieniowych dla pojedynczego pięciominutowego okna czasowego (2019-12-30 21:11:00 - 21:16:00). Ze względu na sliding windows (5-minutowe okna uruchamiane co minutę) oraz nachodzące na siebie micro-batche, surowy skan HBase'a dla tego okna zwrócił 121 wierszy zawierających wielokrotne snapshoty tych samych metryk w różnych momentach czasowych. Poniższa agregacja wykorzystuje dedykowany skrypt Python, który wybiera najnowszy snapshot dla każdej marki/produktu na podstawie stempla czasowego created_at, eliminując duplikaty wynikające z natury przetwarzania strumieniowego.

Pobrano 121 wierszy z HBase dla okna 20191230_2111

=====

OKNO CZASOWE: 20191230_2111

=====

ACTIVE USERS

- 1,142 unique users
- 3,554 total events
- 3.1 events/user avg

PURCHASES

- 56 transactions
- \$17,859.56 total revenue
- \$318.92 avg basket value
- 46 unique buyers

EVENT DISTRIBUTION

- 3,354 views
- 144 carts
- 56 purchases
- Conversion: 4.3% (view→cart)
- Conversion: 38.9% (cart→purchase)

TOP PRODUCTS (top 10 by views - sorted by count)

- Rank #01: samsung 1004767 (45 views, \$238.90)
- Rank #02: apple 1005115 (31 views, \$873.57)
- Rank #03: samsung 1005174 (20 views, \$643.23)
- Rank #04: samsung 1004873 (19 views, \$330.64)
- Rank #05: unknown 26400756 (18 views, \$325.36)
- Rank #06: apple 1002524 (17 views, \$540.04)
- Rank #07: xiaomi 1005253 (17 views, \$264.87)
- Rank #08: huawei 1004903 (17 views, \$76.94)
- Rank #09: apple 4804056 (16 views, \$164.64)
- Rank #10: apple 4804718 (16 views, \$341.58)

TOP BRANDS (top 10 by event count - sorted by count)

- Rank #01: samsung [view] (387 events, \$165,507.15)
- Rank #02: apple [view] (329 events, \$250,306.59)
- Rank #03: xiaomi [view] (235 events, \$46,298.03)

Rysunek 23: Zagregowane wyniki analiz dla okna czasowego 2019-12-30 21:11:00 - 21:16:00

Interpretacja biznesowa metryk

Active Users: Okno 21:11 zarejestrowało 1,142 unikalnych użytkowników generujących 3,554 zdarzeń, co daje średnio 3.1 zdarzenia na użytkownika. Duża liczba użytkowników i zdarzeń sugeruje wzmożoną aktywność na platformie w godzinach wieczornych. Stosunkowo niska liczba zdarzeń na użytkownika wskazuje na schemat postępowania, gdzie użytkownicy przeglądają wiele produktów bez głębokiego zaangażowania.

Purchases: 56 transakcji zakupowych od 46 unikalnych użytkowników generuje łączny przychód \$17,859.56 ze średnią wartością koszyka \$318.92. Zakupowy współczynnik konwersji wynosi $56/3554 \approx 1.6\%$, co jest typową wartością dla e-commerce. Różnica między liczbą transakcji (56) a liczbą kupujących (46) wskazuje że około 10 użytkowników dokonało wielokrotnych zakupów w ramach tego samego pięciominutowego okna, co może sugerować promocje zachęcające do zakupów hurtowych lub użytkowników uzupełniających wcześniej rozpoczęte koszyki.

Event Distribution: Funnel e-commerce pokazuje 3,354 wyświetleń, 144 dodania do koszyka oraz 56 zakupów. Współczynnik konwersji view-to-cart wynosi 4.3%, podczas gdy współczynnik konwersji cart-to-purchase wynosi 38.9%. Wysoki współczynnik cart-to-purchase (prawie 40%) sugeruje, że użytkownicy, którzy dodają produkt do koszyka mają silną intencję zakupu, podczas gdy niski view-to-cart rate wskazuje, że większość użytkowników przegląda produkty bez zaangażowania.

Top Products: Samsung 1004767 zdominował okno z 45 wyświetleniami przy cenie \$238.90, oferując atrakcyjną wartość w kategorii construction.tools.light. Apple 1005115 zajmuje drugie miejsce z 31 wyświetleniami przy znacznie wyższej cenie \$873.57, celując w segment premium. Top 10 pokazuje silną konkurencję między trzema głównymi markami: Samsung (4 produkty w top 10), Apple (4 produkty), Xiaomi (1 produkt) oraz Huawei (1 produkt). Obecność produktu "unknown" 26400756 na piątym miejscu (18 views) wskazuje na luki w danych źródłowych. Rozpiętość cen w top 10 (\$76.94 - \$873.57) odzwierciedla szeroki target demograficzny platformy e-commerce.

Top Brands: Samsung zajmuje pierwsze miejsce z 387 wyświetleniami generującymi \$165,507.15 łącznej wartości produktów, podczas gdy Apple z drugiego miejsca (329 wyświetleń) osiąga wyższą łączną wartość \$250,306.59, co potwierdza pozycjonowanie Apple jako premium brand z wyższymi cenami jednostkowymi. Xiaomi na trzecim miejscu pokazuje 235 wyświetleń z wartością \$46,298.03, odzwierciedlając strategię z niższymi cenami, ale solidną bazą klientów. Analiza pokazuje tylko 3 dominujące marki w top 10, co sugeruje wysoką koncentrację rynku e-commerce wokół głównych producentów elektroniki. Wszystkie pokazane metryki dotyczą event_type 'view' - analiza innych typów zdarzeń (cart, purchase) dla tych samych marek pozwoliłaby na obliczenie brand-specific conversion rates oraz identyfikację, które marki mają najwyższą siłę zakupową a nie tylko klikalność.

Skrypt agregujący dane z HBase

Agregacja danych z 121 surowych wierszy HBase do czytelnej tabeli została zrealizowana przez dedykowany skrypt Python `hbase_window_summary.py`, który implementuje logikę deduplikacji sliding windows. Skrypt łączy się z HBase przez bibliotekę `happybase`,

wykonuje skan dla danego okna czasowego oraz dla każdej marki/produktu wybiera snapshot z najnowszym stemplem czasowym created_at. To eliminuje problem wielokrotnych row keys wynikający z faktu, że streaming job aktualizuje te same metryki w miarę napływania nowych zdarzeń w nachodzących na siebie micro-batchach. Skrypt generuje czytelny output dla konsoli (wykorzystujący ASCII box drawing characters).

7.6. Orkiestracja Spark Streaming przez Apache Airflow

7.6.1. Struktura DAG realtime_analytics_manager

DAG o identyfikatorze realtime_analytics_manager jest odpowiedzialny za zarządzanie Spark Streaming job, który w przeciwieństwie do typowych DAG-ów Airflow nie jest krótkotrwałym zadaniem, ale procesem który powinien działać w sposób ciągły. Rysunek 24 pokazuje kluczowe właściwości DAG-u: schedule interval: co 10 minut, max_active_runs: 1 (co zapobiega wielokrotnym równoczesnym instancjom DAG), catchup: False (co wyłącza historyczne odtwarzanie wykonania) oraz tagi spark, streaming, realtime, hbase kategorizujące DAG w interfejsie Airflow.

```
(airflow-env) bigdatacombains@bigdata-vm:~$ airflow dags details realtime_analytics_manager
/home/win10/airflow-env/lib/python3.10/site-packages/airflow/utils/dot_renderer.py:28 UserWarning: Could not import gra
phviz. Rendering graph to the graphical format will not be possible.
=====
property_name | property_value
=====
dag_id        | realtime_analytics_manager
root_dag_id   | None
is_paused     | False
is_active     | True
is_subdag     | False
last_parsed_time | 2025-12-30T21:29:53.303704+00:00
last_pickled  | None
last_expired  | None
scheduler_lock | None
pickle_id     | None
default_view  | grid
fileloc       | /home/win10/airflow/dags/realtime_analytics_dag.py
file_token    | Ii9ob2l1L3dpbjEwL2FpcmZsb3cvZGFncy9yZWZsdGltZV9hbmFseXRpY3NfZGFuLnB5Ig.OxPVcqP844tN5n
WsMZ39BFXRG1M
owners        | bigdata-team
description   | Zarządza procesem Spark Streaming z HBase
schedule_interval | {'__type': 'CronExpression', 'value': '*/10 * * * *'}
timetable_description | Every 10 minutes
tags          | {'name': 'streaming'}, {'name': 'realtime'}, {'name': 'spark'}, {'name': 'hbase'}
max_active_tasks | 16
max_active_runs | 1
has_task_concurrency_limits | False
has_import_errors | False
next_dagrun    | 2025-12-30T21:20:00+00:00
next_dagrun_data_interval_start | 2025-12-30T21:20:00+00:00
next_dagrun_data_interval_end   | 2025-12-30T21:30:00+00:00
next_dagrun_create_after       | None
```

Rysunek 24: Statystyki działania DAG-u realtime_analytics_manager

Harmonogram co 10 minut nie oznacza że streaming job jest uruchamiany i zatrzymywany co 10 minut – zamiast tego, DAG wykonuje się co 10 minut, aby sprawdzić, czy streaming job nadal działa i w razie potrzeby go uruchomić lub monitorować. To podejście implementuje wzorzec watchdog, gdzie Airflow pełni rolę nadzorca zapewniającego dostępność długotrwałej usługi. Własność next_dagrun: 2025-12-30T21:20:00 wskazuje że następne wykonanie DAG nastąpi o 21:20, co wraz z next_dagrun_data_interval definiującym przedział 21:20-21:30 potwierdza 10-minutową periodyczność.

DAG składa się z 5 zadań wykonywanych w specyficznej sekwencji z rozgałęzieniem: check_if_should_run (ShortCircuitOperator sprawdzający, czy symulator Kafki działa), verify_hbase (weryfikacja dostępności HBase Thrift Server), check_if_running

(BranchPythonOperator decydujący, czy uruchomić nowy streaming job czy tylko monitorować istniejący), oraz dwa alternatywne taski `start_streaming_analytics` i `monitor_health` gdzie tylko jeden jest wykonywany w zależności od decyzji branch operator.

7.6.2. Walidacja wymagań wstępnych

Funkcja `check_if_should_run()` sprawdza, czy symulator Kafki działa, zwracając `True`, jeśli proces istnieje (co powoduje kontynuację DAG-u) lub `False` w przeciwnym razie (co kończy DAG z statusem `success`, ale bez wykonywania dalszych zadań). To zapobiega uruchamianiu streaming job'a, gdy nie ma aktywnej symulacji, oszczędzając zasoby oraz eliminując niepotrzebne error'y wynikające z prób konsumpcji z pustego topiku.

Zadanie `verify_hbase` sprawdza, czy HBase Thrift Server nasłuchuje na porcie 9090, następnie aktywuje środowisko wirtualne Pythona i wykonuje prosty test połączenia przez `happybase.Connection`. Test próbuje otworzyć połączenie oraz zweryfikować istnienie tabeli `realtime_stats`. Test kończy się niepowodzeniem, jeśli tabela nie istnieje. Task ten implementuje strategię wczesnego wykrywania błędów, oszczędzając czas debugowania oraz zapobiegając generowaniu mylących logów.

7.6.3. Logika rozgałęzienia – start vs monitor

Zadanie `check_if_running` wykorzystuje BranchPythonOperator, który zwraca nazwę następnego zadania do wykonania na podstawie logiki w funkcji `check_if_running()`. Funkcja sprawdza, czy istnieje plik PID, odczytuje PID z pliku oraz weryfikuje, czy proces o tym PID faktycznie działa. Jeśli proces działa, funkcja zwraca `'monitor_health'` (co powoduje wykonanie tylko taska monitorującego). Jeśli proces nie działa lub plik PID nie istnieje, funkcja zwraca `'start_streaming_analytics'` (co powoduje uruchomienie nowego streaming job'a).

Dzięki tej logice wielokrotne wykonania nie powodują uruchamiania wielu instancji zadania przesyłania strumieniowego, tylko sprawdzają i w razie potrzeby naprawiają stan systemu. Automatyczne usunięcie starego pliku PID, jeśli proces nie istnieje, zapobiega sytuacji, gdzie zombie PID file blokuje uruchomienie nowego zadania przesyłania strumieniowego po błędzie w poprzednim.

7.6.4. Uruchomienie streaming job'a

Zadanie `start_streaming_analytics` wykorzystuje `nohup` do uruchomienia `spark-submit` w tle z przekierowaniem outputu do pliku log. Komenda tworzy folder do zapisywania checkpointów, aktywuje środowisko wirtualne oraz uruchamia `spark-submit` z konfiguracją `--master local[4]`, `--driver-memory 4g`, `--executor-memory 2g`, oraz `--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3` dla integracji z Kafką.

PID uruchomionego procesu jest zapisywany do pliku PID oraz wyświetlany w logach Airflow wraz ze stemplem czasowym oraz ścieżką do pliku log. Zadanie czeka 15 sekund po uruchomieniu, aby dać czas Sparkowi na inicjalizację, następnie weryfikuje, czy proces nadal działa. Jeśli proces się zatrzymał w ciągu tych 15 sekund, task wyświetla zawartość pliku log oraz kończy się niepowodzeniem, sygnalizując problem wymagający interwencji.

7.6.5. Monitoring health streaming job

Zadanie `monitor_health` jest wykonywane zarówno gdy streaming job był już uruchomiony, jak i po świeżym uruchomieniu przez `start_streaming_analytics`. Zadanie wyświetla ostatnie 10 linii z pliku log streaming job oraz liczy liczbę linii zawierających "Zapisano.*HBase", co daje szybką metrykę aktywności.

7.6.6. Historia wykonań DAG

Rysunek 25 pokazuje historię 14 ostatnich wykonań DAG-u, wszystkie ze statusem success, wykonujące się regularnie co 10 minut. Czas trwania każdego run'u waha się między 12 a 15 sekund dla większości wykonań, z kilkoma wyjątkami trwającymi dłużej. Regularność wykonań potwierdza, że scheduler Airflow działa poprawnie oraz że DAG jest aktywny i monitoruje streaming job zgodnie z harmonogramem.

```
(airflow-env) bigdatacombains@bigdata-vm:~$ airflow dags list-runs -d realtime_analytics_manager --output table
/home/win10/airflow-env/lib/python3.10/site-packages/airflow/utis/dot_renderer.py:28 UserWarning: Could not import gra
phviz. Rendering graph to the graphical format will not be possible.
```

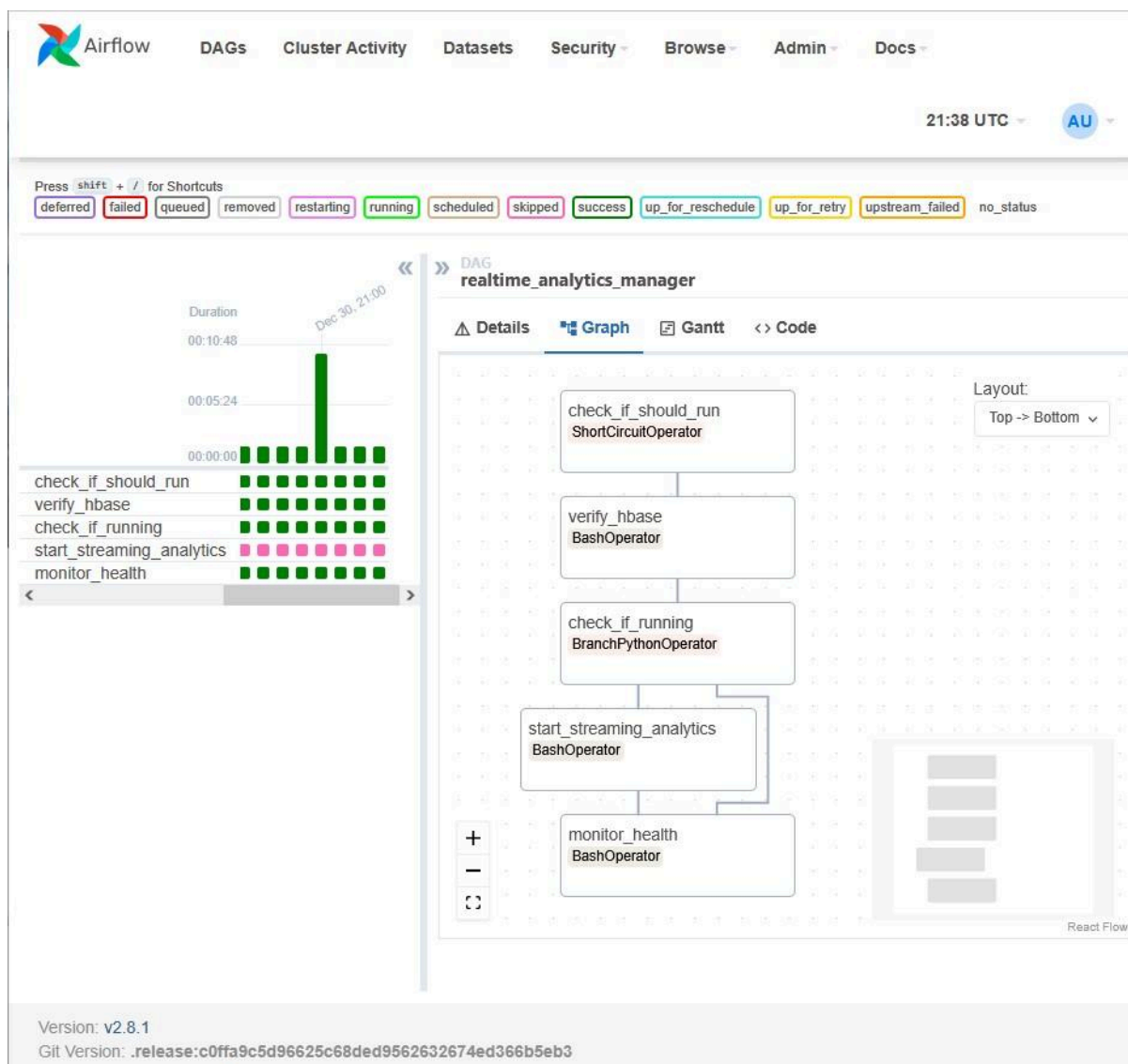
dag_id	run_id	state	execution_date	start_date	end_date
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T21:20:00	2025-12-30T21:30:01	2025-12-30T21:30:13.
anager	30T21:20:00+00:00		+00:00	.058819+00:00	729363+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T21:10:00	2025-12-30T21:20:01	2025-12-30T21:20:13.
anager	30T21:10:00+00:00		+00:00	.475274+00:00	985319+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T21:00:00	2025-12-30T21:10:26	2025-12-30T21:10:39.
anager	30T21:00:00+00:00		+00:00	.019448+00:00	089707+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T20:50:00	2025-12-30T21:00:01	2025-12-30T21:09:52.
anager	30T20:50:00+00:00		+00:00	.847916+00:00	020482+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T20:40:00	2025-12-30T20:50:01	2025-12-30T20:50:15.
anager	30T20:40:00+00:00		+00:00	.418066+00:00	660913+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T20:30:00	2025-12-30T20:40:01	2025-12-30T20:40:23.
anager	30T20:30:00+00:00		+00:00	.915097+00:00	149145+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T20:20:00	2025-12-30T20:30:01	2025-12-30T20:30:14.
anager	30T20:20:00+00:00		+00:00	.354541+00:00	194112+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T20:10:00	2025-12-30T20:20:01	2025-12-30T20:20:13.
anager	30T20:10:00+00:00		+00:00	.413607+00:00	925369+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T20:00:00	2025-12-30T20:10:45	2025-12-30T20:10:58.
anager	30T20:00:00+00:00		+00:00	.171157+00:00	655674+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T19:50:00	2025-12-30T20:00:01	2025-12-30T20:10:08.
anager	30T19:50:00+00:00		+00:00	.533765+00:00	708643+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T19:40:00	2025-12-30T19:50:01	2025-12-30T19:50:15.
anager	30T19:40:00+00:00		+00:00	.563386+00:00	388602+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T19:30:00	2025-12-30T19:40:01	2025-12-30T19:40:14.
anager	30T19:30:00+00:00		+00:00	.268980+00:00	697546+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T19:20:00	2025-12-30T19:30:01	2025-12-30T19:30:14.
anager	30T19:20:00+00:00		+00:00	.817024+00:00	863312+00:00
realtime_analytics_m	scheduled_2025-12-	success	2025-12-30T19:10:00	2025-12-30T19:20:01	2025-12-30T19:20:16.
anager	30T19:10:00+00:00		+00:00	.932043+00:00	503003+00:00

Rysunek 25: Przykładowe wykonania DAG-u `realtime_analytics_manager`

Różnica między `execution_date` a `start_date` jest konsekwentna (około 10 minut + 1 sekunda), co odpowiada semantyce Airflow, gdzie `execution_date` oznacza początek data interval, a faktyczne wykonanie następuje po zakończeniu tego interwału.

7.6.7. Airflow UI – wizualizacja DAG

Rysunek 26 prezentuje wizualną reprezentację DAG-u w interfejsie webowym Airflow, pokazującą 5 zadań z zależnościami oraz historię wykonań w formie grid view. Szczególnie interesujące są różowe (skipped) wykonania zadania `start_streaming_analytics` w ostatnich uruchomieniach, co oznacza że task został pominięty przez branch operator – streaming job już działał, więc `check_if_running` zdecydował o wykonaniu `monitor_health` zamiast uruchamiania nowego job'a. Różowy kolor w Airflow UI oznacza że task został celowo pominięty.



Rysunek 26: Graf w interfejsie graficznym Airflow reprezentujący DAG realtime_analytics_manager

Ta wizualizacja potwierdza poprawne działanie logiki branching: w pierwszym wykonaniu, start_streaming_analytics miał status zielony (success), a w kolejnych 10-minutowych checkach task ten był różowy, ponieważ streaming job nadal działał nieprzerwanie. Task monitor_health ma zielony status we wszystkich ostatnich wykonaniach, potwierdzając, że monitoring działa poprawnie oraz że streaming job jest w dobrej kondycji przez ostatnie kilka godzin. Sekwencja różowych start_streaming_analytics w połączeniu z zielonymi monitor_health pokazuje wzorcowe działanie long-running job manager: DAG sprawdza stan co 10 minut, wykrywa że proces działa i wykonuje tylko monitoring zamiast próbować uruchomić drugi proces.

8. Analiza wsadowa Batch Layer

Przetwarzanie wsadowe w architekturze Lambda stanowi rdzeń głębokiej analizy danych historycznych. W przeciwieństwie do Speed Layer, który koncentruje się na przetwarzaniu w czasie rzeczywistym, Batch Layer umożliwia przeprowadzanie złożonych analiz statystycznych na większych zbiorach danych, generując wnioski biznesowe o wysokiej wartości dla działów analitycznych oraz marketingowych platformy e-commerce.

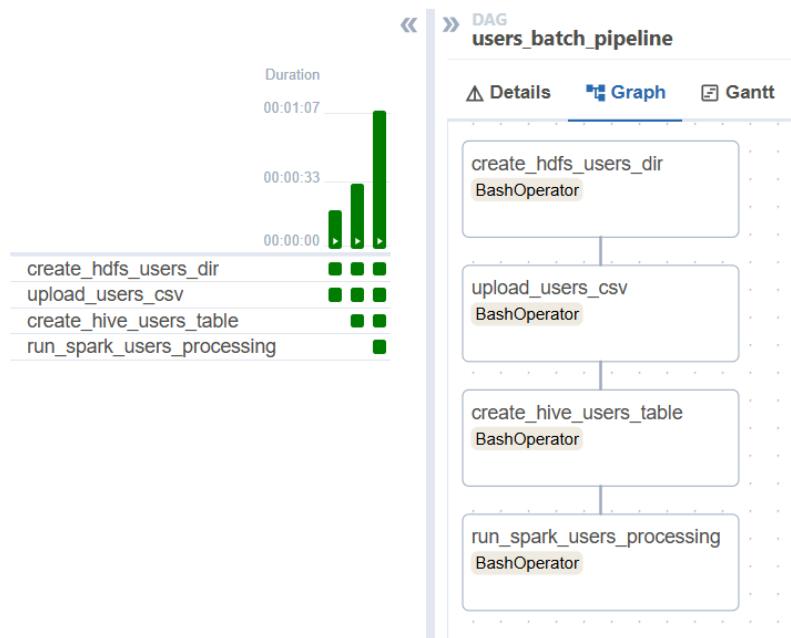
System realizuje trzy kluczowe analizy wsadowe: lejek zakupowy (funnel analysis), rekomendacje demograficzne oraz analizę efektywności marek. Każda z tych analiz została zaimplementowana jako osobny skrypt PySpark, orkiestrowany przez dedykowany DAG w Apache Airflow, działający zgodnie z harmonogramem dostosowanym do analizy.

8.1. Przepływ danych użytkowników

Przepływ realizuje kompleksowy proces ETL dla danych demograficznych z pliku users.csv (5 mln rekordów, 370 MB) opisanego w rozdziale 3.3, przekształcając surowe profile we wzbogacone zbiory analityczne wykorzystywane przez system rekomendacji.

8.1.1. Architektura DAG w Apache Airflow

DAG users_batch_pipeline został skonfigurowany bez automatycznego harmonogramu (schedule_interval=None, catchup=False), uruchamiany ręcznie w zależności od dostępności nowych danych. Przepływ składa się z sekwencyjnych zadań: utworzenie katalogu w HDFS → załadowanie pliku → rejestracja w Hive → transformacje w Spark.



Rysunek 27: Zrzut ekranu Airflow Graph View pokazujący DAG users_batch_pipeline z czterema zadaniami i historią wykonań

8.1.2. Załadunek danych do HDFS

Zadanie create_hdfs_users_dir tworzy katalog /raw/users w HDFS. Zadanie upload_users_csv transferuje plik z lokalizacji /home/win10/users.csv do lokalizacji

/raw/users/users.csv w HDFS z zabezpieczeniem przed nadpisaniem. Transfer wykorzystuje natywny protokół HDFS z replikacją równą 2, zapewniając odporność na awarie.

```
bigdatacombains@bigdata-vm:~$ hdfs dfs -ls -h /raw/users
Found 1 items
-rw-r--r-- 1 bigdatacombains supergroup 361.2 M 2025-12-29 13:11 /raw/users/users.csv
```

Rysunek 28: Wynik komendy `hdfs dfs -ls -h /raw/users` pokazujący plik `users.csv` z rozmiarem i datą utworzenia

8.1.3. Rejestracja w Apache Hive

Tabela `users_raw` została zdefiniowana jako tabela zewnętrzna w pliku w lokalizacji `/home/win10/sql/hive/tables/users_raw.hql`, wskazująca na lokalizację `/raw/users` w HDFS. Schemat obejmuje osiem kolumn. Kluczowa właściwość pomijająca wiersz nagłówka eliminuje błędy parsowania podczas odczytu pliku CSV. Zadanie `create_hive_users_table` wykonuje definicję DDL, zapewniając automatyczną rejestrację w Hive Metastore zintegrowanym ze Spark SQL.

8.1.4. Transformacje w Apache Spark

Zadanie Spark zapisane w pliku `/home/win10/sql/spark/users_processing.py` wykonuje trzy transformacje na danych odczytanych z Hive:

Obliczanie wieku: Kolumna `age` jest generowana jako różnica między rokiem bieżącym a rokiem urodzenia. Dynamiczne obliczanie zapewnia aktualność segmentacji bez modyfikacji danych źródłowych.

Klasyfikacja grup wiekowych: Segmentacja użytkowników do czterech kategorii (18-25, 26-35, 36-50, 50+) stosowanych w analizie rekomendacji. Konwersja z metryki ciągłej do kategoriycznej zwiększa siłę statystyczną analiz.

Agregacja demograficzna: Grupowanie według miasta i grupy wiekowej redukuje 5 mln rekordów do kilku tysięcy segmentów demograficznych, drastycznie przyspieszając późniejsze zapytania analityczne.

Wyniki są zapisywane do `/processed/user_demographics` w formacie Parquet. Zadanie `run_spark_users_processing` uruchamia job w trybie lokalnym.

8.2. Przepływ danych produktów

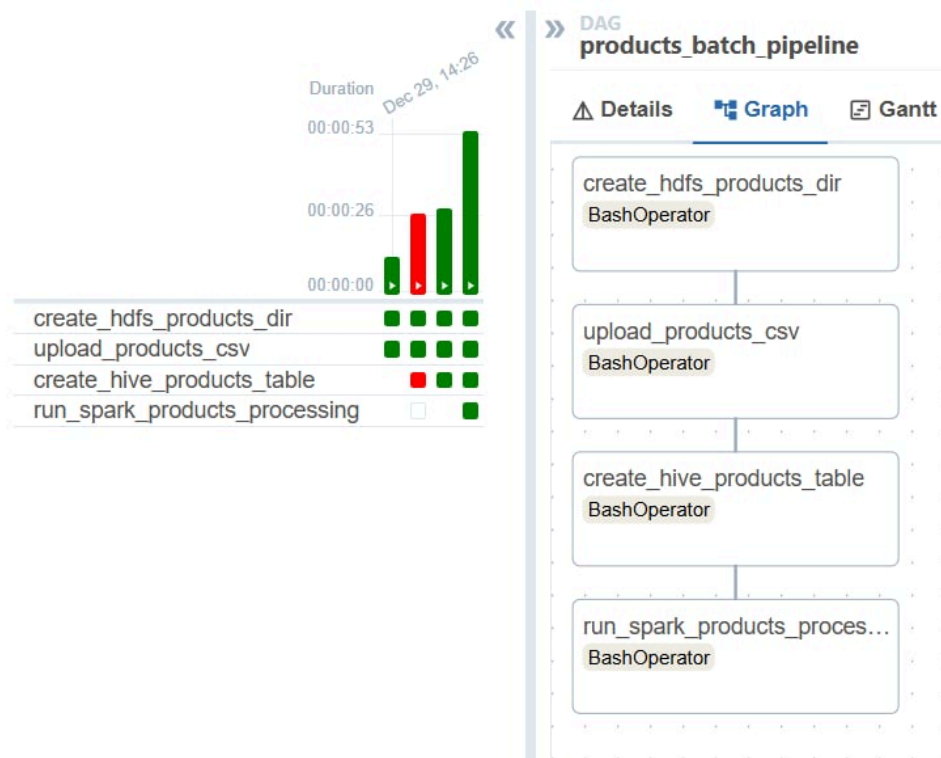
Przepływ `products_batch_pipeline` realizuje analogiczny proces ETL dla katalogu produktów, wygenerowanego przez model językowy na podstawie danych strumieniowych.

8.2.1. Architektura DAG w Apache Airflow

DAG `products_batch_pipeline` został skonfigurowany z identycznymi parametrami jak przepływ użytkowników – bez automatycznego harmonogramu (`schedule_interval=None`, `catchup=False`), co zapewnia uruchamianie ręczne w zależności od dostępności nowych danych źródłowych. Przepływ składa się z czterech sekwencyjnie wykonywanych zadań

realizujących wzorzec ETL: utworzenie katalogu w HDFS → załadunek pliku → rejestracja w Hive → transformacje w Spark.

Wykorzystanie identycznej architektury dla obu przepływów (użytkowników i produktów) zapewnia spójność implementacji oraz ułatwia utrzymanie systemu. Operator bash jest stosowany konsekwentnie dla wszystkich zadań, umożliwiając bezpośrednią interakcję z komponentami ekosystemu Apache.



Rysunek 29: Zrzut ekranu Airflow Graph View pokazujący DAG products_batch_pipeline z czterema zadaniami i historią wykonań

8.2.2. Załadunek do HDFS

Zadanie create_hdfs_products_dir tworzy katalog /raw/products w HDFS. Następnie zadanie upload_products_csv transferuje plik /home/win10/products.csv do lokalizacji /raw/products/products.csv w HDFS z zabezpieczeniem przed nadpisaniem danych.

```
bigdatacombains@bigdata-vm:~$ hdfs dfs -ls -h /raw/products
Found 1 items
-rw-r--r--  1 bigdatacombains supergroup    19.4 M 2025-12-29 14:27 /raw/products/products.csv
```

Rysunek 30: Wynik komendy hdfs dfs -ls -h /raw/users pokazujący plik users.csv z rozmiarem i datą utworzenia

8.2.3. Rejestracja w Hive

Tabela products_raw definiuje sześć kolumn: identyfikator produktu, nazwa produktu, identyfikator kategorii, nazwa kategorii, kod kategorii oraz marka. Następnie zadanie create_hive_products_table wykonuje plik /home/win10/sql/hive/tables/products_raw.hql, rejestrując tabelę zewnętrzną w Hive Metastore.

8.2.4. Transformacje w Spark

Zadanie `run_spark_products_processing` uruchamia job zapisany w pliku `/home/win10/sql/spark/products_processing.py`. Skrypt wykonuje filtrację rekordów z brakującymi wartościami w polach `cena` oraz `kod kategorii`, zapewniając kompletność danych w warstwie przetworzonych agregatów. Dane są zapisywane do `/processed/products` w formacie Parquet oraz z partycjonowaniem według kodu kategorii, co umożliwia efektywne zapytania filtrujące według kategorii produktowych w analizach.

8.3. Orkiestracja analiz wsadowych

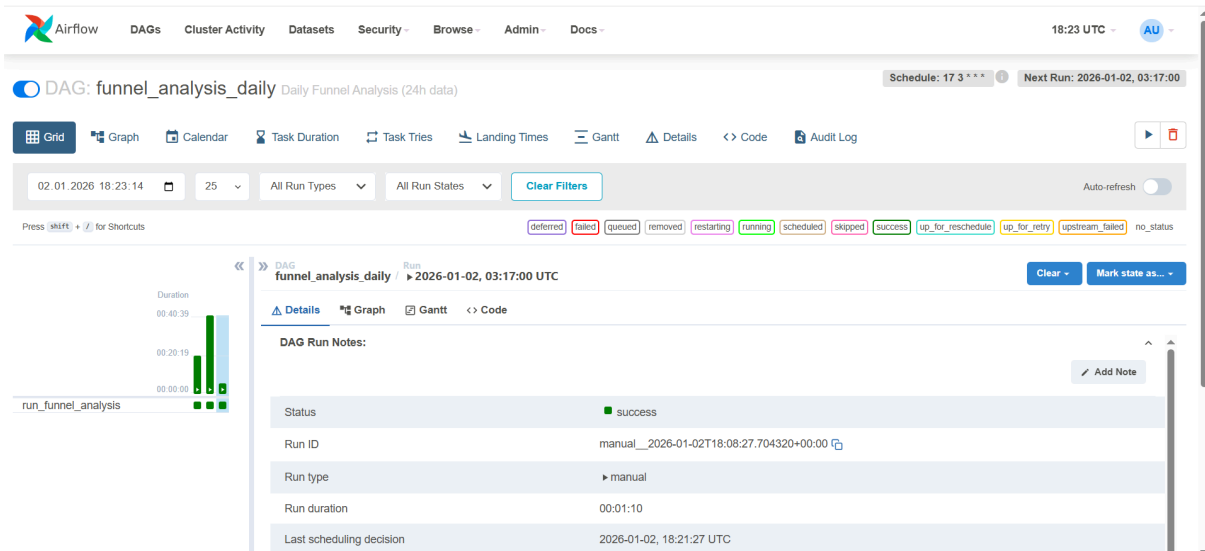
8.3.1. DAG `funnel_analysis_daily`

Pierwszy przepływ odpowiada za codzienną analizę lejka zakupowego. DAG `funnel_analysis_daily` uruchamia się o godzinie 3:17 AM każdego dnia, przetwarzając dane z ostatnich 24 godzin. Wybór tej godziny nie jest przypadkowy – system operuje na danych z ostatniej doby, a uruchomienie analizy w godzinach nocnych minimalizuje wpływ na inne procesy systemowe.

```
(airflow-env) bigdatacombains@bigdata-vm:/home/win10$ airflow dags details funnel_analysis_daily
/home/win10/airflow-env/lib/python3.10/site-packages/airflow/utils/dot_renderer.py:28 UserWarning: Could not import graphviz. Rendering graph to the graphical format will not be possible.
=====
property_name | property_value
=====+=====
dag_id        | funnel_analysis_daily
root_dag_id   | None
is_paused     | False
is_active     | True
is_subdag     | False
last_parsed_time | 2026-01-02T17:45:08.745701+00:00
last_pickled  | None
last_expired  | None
scheduler_lock | None
pickle_id     | None
default_view  | grid
fileloc       | /home/win10/airflow/dags/funnel_analysis_dag.py
file_token    | Ii9ob2llL3dpbjEwL2FpcmZsb3cvZGFncy9mdW5uZWxfYW5hbHlzaXNfZGFuLnB5Ig.eCWgODtAWXb
              | r9PACgdDEh5mAu5k
owners       | bigdatacombains
description   | Daily Funnel Analysis (24h data)
schedule_interval | {'__type': 'CronExpression', 'value': '17 3 * * *'}
timetable_description | At 03:17
tags         | {'name': 'funnel'}, {'name': 'daily'}, {'name': 'analytics'}
max_active_tasks | 16
max_active_runs | 16
has_task_concurrency_limits | False
has_import_errors | False
next_dagrun    | 2026-01-02T03:17:00+00:00
next_dagrun_data_interval_start | 2026-01-02T03:17:00+00:00
next_dagrun_data_interval_end   | 2026-01-03T03:17:00+00:00
next_dagrun_create_after       | 2026-01-03T03:17:00+00:00
=====
```

Rysunek 31: Szczegółowe informacje na temat DAGu `funnel_analysis_daily`

Rysunek z interfejsu Airflow przedstawia konfigurację DAG-u. Schedule interval `17 3 * * *` oznacza wykonanie o 3:17 każdego dnia. Własność `max_active_runs: 16` pozwala na równoległe wykonanie maksymalnie 16 instancji DAG, choć w praktyce działa tylko jedna na raz ze względu na harmonogram. Tag `funnel` oraz `daily` ułatwiają kategoryzację i wyszukiwanie w interfejsie Airflow.



Rysunek 32: Widok w Airflow UI dla DAGu `funnel_analysis_daily`

Przykładowe uruchomienie z 2 stycznia 2026 trwało 1 minutę i 10 sekund, zakańczając się sukcesem. Czas wykonania jest stosunkowo krótki ze względu na efektywne przetwarzanie przez Spark oraz ograniczenie zakresu czasowego do ostatnich 24 godzin.

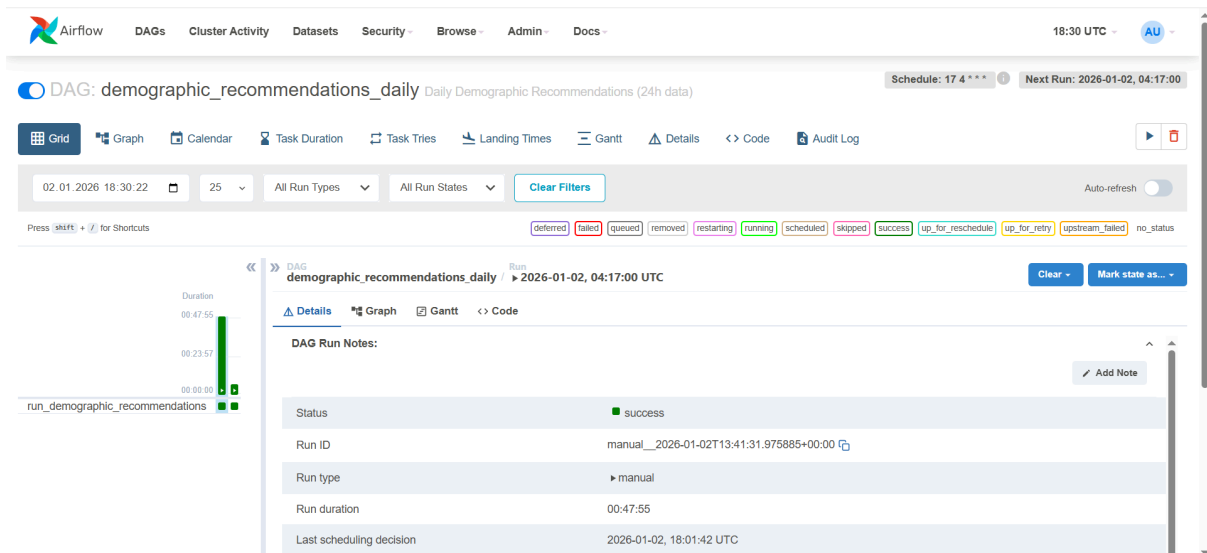
8.3.2. DAG `demographic_recommendations_daily`

Drugi przepływ generuje spersonalizowane rekomendacje produktowe na podstawie profili demograficznych użytkowników. DAG `demographic_recommendations_daily` uruchamia się o 4:17 AM, czyli godzinę po zakończeniu analizy lejka. Schedule interval `17 4 * * *` zapewnia sekwencyjne wykonanie analiz, minimalizując ryzyko przeciążenia klastra przez Spark.

```
(airflow-env) bigdatacombains@bigdata-vm:/home/win10$ airflow dags details demographic_recommendations_daily
/home/win10/airflow-env/lib/python3.10/site-packages/airflow/utils/dot_renderer.py:28 UserWarning: Could not import graphviz. Rendering graph to the graphical format will not be possible.
property_name | property_value
-----|-----
dag_id | demographic_recommendations_daily
root_dag_id | None
is_paused | False
is_active | True
is_subdag | False
last_parsed_time | 2026-01-02T17:45:07.359381+00:00
last_pickled | None
last_expired | None
scheduler_lock | None
pickle_id | None
default_view | grid
fileloc | /home/win10/airflow/dags/demographic_recommendations_dag.py
file_token | Ii9ob21lL3dpcjEwL2FpcmZsb3cvZGFncy9kZW1vZ3JhcGhpY19yZWVnbWllbmRhdGlvdnNfZGFuLn
owners | bigdatacombains
description | Daily Demographic Recommendations (24h data)
schedule_interval | {'__type': 'CronExpression', 'value': '17 4 * * *'}
timetable_description | At 04:17
tags | {'name': 'recommendations'}, {'name': 'daily'}, {'name': 'analytics'}
max_active_tasks | 16
max_active_runs | 16
has_task_concurrency_limits | False
has_import_errors | False
next_dagrun | 2026-01-02T04:17:00+00:00
next_dagrun_data_interval_start | 2026-01-02T04:17:00+00:00
next_dagrun_data_interval_end | 2026-01-03T04:17:00+00:00
next_dagrun_create_after | 2026-01-03T04:17:00+00:00
```

Rysunek 33: Szczegółowe informacje na temat DAGu `demographic_recommendations_daily`

Analiza przetwarza dane eventów z ostatnich 24 godzin, łącząc je z katalogiem produktów oraz profilami użytkowników zawierającymi informacje demograficzne (wiek, region, miasto). Wyniki są segmentowane według grup wiekowych (18-24, 25-34, 35-44, 45-54, 55+), a dla każdego segmentu system oblicza top 10 produktów według złożonego wskaźnika recommendation_score.



Rysunek 34: Widok w Airflow UI dla DAGu demographic_recommendations_daily

Przykładowe uruchomienie z 2 stycznia 2026 trwało 47 minut i 55 sekund. Dłuższy czas wykonania wynika z konieczności wykonania trzech operacji JOIN (events + users + products) oraz złożonych agregacji grupujących dane według wielu wymiarów.

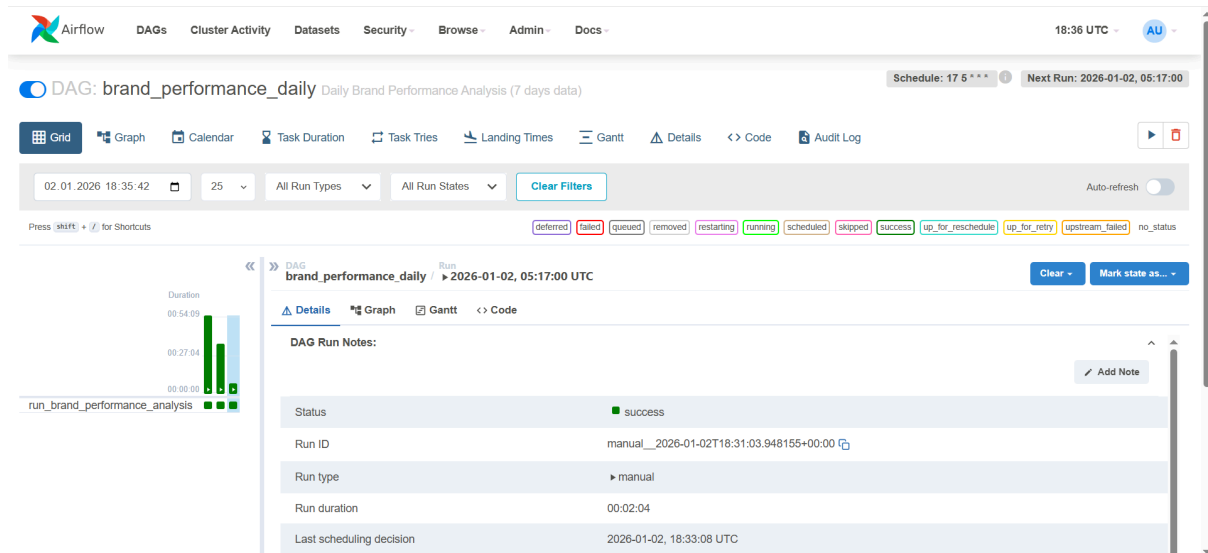
8.3.3. DAG brand_performance_daily

Trzeci przepływ analizuje efektywność marek w kontekście demografii użytkowników. DAG brand_performance_daily wykonuje się o 5:17 AM, przetwarzając dane z ostatnich 7 dni. Dłuższy horyzont czasowy (tydzień zamiast doby) wynika z potrzeby zgromadzenia wystarczającej ilości danych dla rzadziej występujących marek.

```
(airflow-env) bigdatacombains@bigdata-vm:/home/win10/airflow/dags$ airflow dags details brand_performance_daily
/home/win10/airflow-env/lib/python3.10/site-packages/airflow/utils/dot_renderer.py:28 UserWarning: Could not import graphviz. Rendering graph to the graphical format will not be possible.
=====
property_name | property_value
=====+=====
dag_id        | brand_performance_daily
root_dag_id   | None
is_paused     | False
is_active     | True
is_subdag     | False
last_parsed_time | 2026-01-02T17:06:30.750835+00:00
last_pickled  | None
last_expired  | None
scheduler_lock | None
pickle_id     | None
default_view  | grid
fileloc       | /home/win10/airflow/dags/brand_performance_dag.py
file_token    | Ii9ob2llL3dpbjEwL2FpcmZsb3cvZGFncy9icmFuZGF9wZlJmb3JtYW5jZW9kYWcucHki.ZiHveV-pg8QfAfErtZiC9Hva79k
owners       | bigdatacombains
description   | Daily Brand Performance Analysis (7 days data)
schedule_interval | {'_type': 'CronExpression', 'value': '17 5 * * *'}
timetable_description | At 05:17
tags         | ({'name': 'brand'}, {'name': 'daily'}, {'name': 'analytics'})
max_active_tasks | 16
max_active_runs | 16
has_task_concurrency_limits | False
has_import_errors | False
next_dagrun   | 2026-01-02T05:17:00+00:00
next_dagrun_data_interval_start | 2026-01-02T05:17:00+00:00
next_dagrun_data_interval_end   | 2026-01-03T05:17:00+00:00
next_dagrun_create_after       | 2026-01-03T05:17:00+00:00
```

Rysunek 35: Szczegółowe informacje na temat DAGu brand_performance_daily

Schedule interval 17 5 * * * umieszcza tę analizę jako ostatnią w sekwencji nocnych zadań. Tagi brand oraz daily (pomimo tygodniowego zakresu danych, analiza wykonuje się codziennie) pomagają w identyfikacji przepływu.



Rysunek 36: Widok w Airflow UI dla DAGu brand_performance_daily

Przykładowe uruchomienie z 2 stycznia 2026 trwało 2 minuty i 4 sekundy. Stosunkowo krótki czas wykonania, pomimo 7-dniowego zakresu danych, wynika z optymalizacji polegającej na zapisie jedynie top 100 marek według przychodu, co radykalnie zmniejsza liczbę rekordów zapisywanych do HBase.

8.4. Analiza lejka zakupowego (Funnel Analysis)

8.4.1. Cel analizy

Analiza lejka zakupowego identyfikuje momenty, w których użytkownicy porzucają proces zakupowy, umożliwiając optymalizację procesu sprzedaży. Ścieżka zakupowa składa się z trzech etapów: view (wyświetlenie produktu), cart (dodanie do koszyka) oraz purchase (zakup).

System oblicza kluczowe metryki dla każdego etapu lejka:

- Liczba użytkowników, którzy dotarli do danego etapu
- Współczynniki konwersji między etapami (view→cart, cart→purchase, view→purchase)
- Drop-off rates (wskaźniki porzuceń) między etapami
- Średni czas między kolejnymi etapami (w sekundach)

Wszystkie metryki są obliczane z podziałem na kategorie produktów oraz marki, co pozwala na identyfikację segmentów wymagających optymalizacji.

8.4.2. Implementacja

Skrypt `funnel_analysis.py` wykorzystuje Apache Spark do przetwarzania danych z ostatnich 24 godzin. Kluczowym elementem implementacji było poprawne określenie zakresu czasowego analizy – system musi uwzględniać rzeczywisty czas wystąpienia zdarzeń, a nie czas ich zapisania w systemie.

Dane są pobierane z tabeli `user_events` przechowywanej w Hive, następnie filtrowane według kolumny `mapped_time`, która reprezentuje rzeczywisty czas wystąpienia zdarzenia po przeskalowaniu czasowym z 2019 na lata 2025/2026.

Proces obliczania metryk lejka przebiega w kilku etapach:

1. Agregacja zdarzeń użytkownika: System identyfikuje pierwszą interakcję użytkownika z każdym produktem na każdym etapie lejka (`view`, `cart`, `purchase`), zapisując dokładny czas tych zdarzeń.
2. Obliczenie czasów między etapami: Na podstawie różnic między czasami poszczególnych etapów system wyznacza średni czas potrzebny użytkownikom na przejście między kolejnymi krokami procesu zakupowego.
3. Agregacja według segmentów: Dane są grupowane według kategorii produktów oraz marek, z obliczeniem liczby unikalnych użytkowników na każdym etapie oraz średnich czasów przejścia.
4. Obliczenie współczynników: System automatycznie wylicza wskaźniki konwersji (ile procent użytkowników przechodzi z jednego etapu do następnego) oraz drop-off rates (ile procent użytkowników porzuca proces na danym etapie).

Wszystkie wartości są konwertowane do formatu tekstowego z odpowiednią precyzją przed zapisem do HBase, co eliminuje problemy z typami danych.

8.4.3. Wyniki analizy

Jupyter notebook `ecommerce_analytics.ipynb` zawiera szczegółową wizualizację wyników analizy lejka. System przetwarza dane z 13,258 rekordów reprezentujących metryki lejka dla różnych kategorii i marek.

Według kategorii produktów:

Najwyższy ruch obserwowany jest w kategoriach `electronics` (elektronika), `furniture` (meble) oraz `appliances` (AGD). Wyniki pokazują wyraźne różnice w zachowaniach użytkowników w zależności od typu produktu.

Największy drop-off występuje między etapami `view`→`cart`, gdzie średnio tylko 15-20% użytkowników dodaje produkt do koszyka. To kluczowy moment wymagający optymalizacji – większość potencjalnych klientów rezygnuje już na tym etapie, prawdopodobnie z powodu ceny, braku przekonujących informacji o produkcie lub problemów z zaufaniem do platformy.

Konwersja `cart`→`purchase` jest znacząco wyższa (40-60%), co sugeruje że użytkownicy dodający produkty do koszyka mają silną intencję zakupową. Po podjęciu decyzji o dodaniu produktu do koszyka, większość użytkowników finalizuje transakcję, co wskazuje na stosunkowo sprawny proces checkout.

Średni czas między view a cart wynosi od kilku minut do kilkunastu minut, w zależności od kategorii. Produkty drogie lub wymagające przemyślenia (np. elektronika) charakteryzują się dłuższym czasem decyzji, podczas gdy produkty codziennego użytku są dodawane do koszyka szybciej.

Najszybsze decyzje zakupowe (najkrótszy czas view→purchase) obserwowane są w kategoriach FMCG (fast-moving consumer goods). To potwierdza impulsywny charakter zakupów produktów codziennego użytku.

Według marek:

Top marki (Apple, Samsung, Sony) charakteryzują się wyższymi współczynnikami konwersji niż średnia. Silna rozpoznawalność marki i zaufanie konsumentów przekładają się bezpośrednio na wyższe współczynniki finalizacji zakupów.

Marki premium wykazują dłuższy średni czas między view a purchase, co sugeruje bardziej przemyślane decyzje zakupowe. Klienci porównują oferty, czytają opinie i zastanawiają się nad inwestycją przed podjęciem decyzji o zakupie drogiego produktu.

Niektóre marki o wysokiej liczbie wyświetleń mają nieproporcjonalnie niskie współczynniki konwersji, wskazując na potencjalne problemy z prezentacją produktu lub ceną. To sygnał dla działów marketingowych o konieczności interwencji.

Identyfikacja problemów:

System automatycznie wykrywa kategorie oraz marki wymagające interwencji marketingowej. Produkty z wysokim view, ale niskim cart mogą wymagać lepszych opisów, bardziej atrakcyjnych zdjęć lub bardziej konkurencyjnych cen. Z drugiej strony, produkty z wysokim cart, ale niskim purchase mogą mieć problemy z procesem finalizacji zamówienia, nieoczekiwanymi kosztami wysyłki lub zbyt skomplikowanym formularzem zamówienia.

8.4.4. Zapis wyników do HBase

Wyniki analizy są zapisywane do tabeli funnel_analytics w HBase, wykorzystującej dwa rodzaje grup kolumn (column families):

- segments: Przechowuje metadane dotyczące typu segmentu (kategoria lub marka), nazwy segmentu oraz czasu utworzenia rekordu
- stats: Zawiera wszystkie metryki liczbowe, w tym liczby użytkowników na każdym etapie, współczynniki konwersji, wskaźniki porzuceń oraz średnie czasy przejścia między etapami

Klucz wiersza (row key) ma format timestamp_type_name, na przykład 20260102_031700_category_electronics.smartphone. Taki format zapewnia kilka korzyści: umożliwia efektywne skanowanie rekordów według czasu, ułatwia filtrowanie według typu segmentu (kategoria vs marka) oraz gwarantuje unikalność kluczy.

Każdy rekord zawiera kompletny zestaw metryk, co eliminuje konieczność ponownego obliczania wartości podczas odczytu danych. Aplikacje końcowe mogą pobrać wszystkie informacje o danym segmencie jednym zapytaniem do bazy danych.

8.5. Rekomendacje demograficzne

8.5.1. Cel analizy

System rekomendacji produktowych wykorzystuje profil demograficzny użytkownika do personalizacji doświadczeń zakupowych.

Założenie biznesowe jest proste: użytkownicy w podobnym wieku i z tego samego regionu mają podobne preferencje zakupowe. System identyfikuje produkty najbardziej popularne w danym segmencie demograficznym i rekomenduje je innym użytkownikom należącym do tego samego segmentu.

8.5.2. Implementacja

Proces analizy składa się z czterech głównych kroków:

Krok 1: Wczytanie i wzbogacenie danych

System pobiera dane z trzech źródeł: zdarzenia użytkowników z ostatnich 24 godzin (przechowywane w tabeli Hive), profile użytkowników zawierające informacje demograficzne oraz katalog produktów z nazwami i opisami.

Profil każdego użytkownika jest automatycznie wzbogacany o wiek obliczony na podstawie daty urodzenia. Następnie użytkownicy są przypisywani do jednej z pięciu grup wiekowych: 18-24, 25-34, 35-44, 45-54 oraz 55+. Dodatkowo system przypisuje każdemu użytkownikowi region geograficzny.

Krok 2: Łączenie informacji

System łączy zdarzenia użytkowników z ich profilami demograficznymi, wzbogacając każde zdarzenie (view, cart, purchase) o informacje o wieku i lokalizacji użytkownika. Po połączeniu danych, system generuje statystyki aktywności poszczególnych segmentów, identyfikując najbardziej aktywne grupy wiekowe na platformie.

Krok 3: Obliczenie wskaźnika rekomendacji

Dla każdej kombinacji grupy wiekowej i produktu system oblicza cztery podstawowe metryki:

Liczba unikalnych użytkowników, którzy wyświetlili produkt

Liczba unikalnych użytkowników, którzy dodali produkt do koszyka

Liczba unikalnych użytkowników, którzy zakupili produkt

Suma przychodów wygenerowanych przez sprzedaż tego produktu

Ostatecznie dla każdego segmentu wybieranych jest top 10 produktów o najwyższym wskaźniku rekomendacji. Te produkty stanowią personalizowane rekomendacje dla użytkowników z danego segmentu demograficznego.

8.5.3. Wyniki analizy

Jupyter notebook prezentuje szczegółowe wyniki działania systemu rekomendacji. Analiza obejmuje 100 grup demograficznych, łącznie obejmując 306 unikalnych produktów w rekomendacjach.

Najczęściej rekomendowane produkty:

Niektóre produkty pojawiają się w rekomendacjach wielu segmentów demograficznych. Na przykład work lights Model 1188 występuje w rekomendacjach aż 29 różnych segmentów, co wskazuje na jego uniwersalną popularność. Fishing equipment Model 101268 pojawia się w 24 segmentach. Produkty elektroniczne, takie jak smartfony i laptopy, dominują w rekomendacjach, co odzwierciedla ich wysoką popularność wśród użytkowników platformy.

Skuteczność według pozycji w rankingu:

Analiza pokazuje wyraźny spadek zaangażowania użytkowników w miarę przesuwania się w dół listy rekomendacji. Pierwszy produkt w rekomendacji (rank 1) generuje średnio 1,775 wyświetleń i 17 dodań do koszyka. Piąty produkt (rank 5) notuje już tylko 942 wyświetlenia i 4 dodania do koszyka. Ostatni produkt w top 10 (rank 10) uzyskuje zaledwie 626 wyświetleń i 0 dodań do koszyka.

Spadek aktywności z pozycji 1 do 10 wynosi 64.7%, co jednoznacznie sugeruje, że użytkownicy koncentrują się głównie na pierwszych pozycjach rekomendacji. To istotna informacja dla projektantów interfejsu – pierwsze pozycje mają kluczowe znaczenie.

Preferencje według grup wiekowych:

System wykrywa wyraźne różnice w preferencjach między grupami wiekowymi, co potwierdza zasadność segmentacji demograficznej:

- Użytkownicy w wieku 18-24 lat preferują produkty elektroniczne, szczególnie smartfony i gaming equipment
- Grupa 25-34 lat wykazuje zainteresowanie produktami dla domu, takimi jak meble i AGD
- Użytkownicy w wieku 35-44 lat wybierają mix elektroniki i produktów lifestyle
- Starsi użytkownicy (45-54 lat) koncentrują się na AGD i narzędziach
- Najstarsza grupa (55+) preferuje produkty zdrowotne i AGD

Te wzorce są zgodne z intuicją biznesową – młodszy użytkownicy interesują się technologią, podczas gdy starsi skupiają się na produktach praktycznych i domowych.

8.5.4. Zapis wyników do HBase

Wyniki są zapisywane do tabeli `personalized_recommendations` w HBase. Klucz wiersza ma format `timestamp_segment_age_group`, na przykład `20260102_041700_segment_25-34`.

Struktura rekordu wykorzystuje dwa rodzaje grup kolumn:

Column family: `profile` - zawiera metadane segmentu, w tym grupę wiekową, typ segmentacji (zawsze "demographic" w tym przypadku) oraz timestamp utworzenia rekomendacji.

Column family: `products` - przechowuje szczegóły rekomendowanych produktów. Dla każdej pozycji w rankingu (od 1 do 10) zapisywane są cztery informacje: identyfikator produktu, nazwa produktu, wartość `recommendation score` oraz liczba wyświetleń.

Przykładowo dla pierwszej pozycji w rankingu system zapisuje kolumny: `rank_1_product_id`, `rank_1_product_name`, `rank_1_score` oraz `rank_1_views`. Dzięki takiej strukturze aplikacja kliencka może pobrać wszystkie rekomendacje dla danego segmentu jednym zapytaniem do bazy danych, co zapewnia wysoką wydajność systemu.

8.6. Analiza efektywności marek

8.6.1. Cel analizy

Analiza efektywności marek dostarcza informacji o tym, które marki generują największą wartość dla platformy e-commerce oraz jak różne grupy demograficzne reagują na poszczególne marki. Działy marketingowe mogą wykorzystać te dane do optymalizacji współpracy z producentami, negocjacji warunków handlowych oraz precyzyjnego targetowania kampanii reklamowych.

System monitoruje pięć kluczowych metryk dla top 100 marek według przychodu:

- Unique users: Liczba unikalnych użytkowników, którzy mieli jakąkolwiek interakcję z produktami danej marki (wyświetlenie, dodanie do koszyka lub zakup)
- Total views: Łączna liczba wyświetleń wszystkich produktów marki
- Total purchases: Łączna liczba zakupów produktów marki
- Total revenue: Całkowity przychód wygenerowany przez markę w analizowanym okresie
- Overall conversion rate: Stosunek liczby zakupów do liczby wyświetleń, wyrażony jako procent

Wszystkie metryki są obliczane z podziałem na grupy wiekowe, co pozwala na identyfikację segmentów demograficznych najbardziej zainteresowanych daną marką.

8.6.2. Implementacja

Skrypt `brand_performance_analysis.py` przetwarza dane z ostatnich 7 dni, co zapewnia wystarczającą ilość danych nawet dla rzadziej występujących marek. Dłuższy horyzont czasowy (tydzień zamiast jednego dnia) jest niezbędny, ponieważ niektóre marki mogą nie generować wystarczającej liczby zdarzeń dziennie, aby analiza była statystycznie wiarygodna.

Filtrowanie i przygotowanie danych:

System najpierw filtruje tylko te zdarzenia, które mają przypisaną markę, ponieważ nie wszystkie produkty w katalogu mają zdefiniowane pole `brand`. Następnie dane są ograniczane do ostatnich 7 dni na podstawie rzeczywistego czasu wystąpienia zdarzenia (kolumna `mapped_time`).

Wzbogacanie o dane demograficzne:

Zdarzenia są łączone z profilem użytkowników, aby wzbogacić je o informacje demograficzne. System wykorzystuje `left join`, co zapewnia że zachowane są wszystkie zdarzenia, nawet jeśli dla niektórych użytkowników brakuje danych demograficznych. Grupy wiekowe są obliczane automatycznie na podstawie wieku użytkownika, przypisując każdego do jednej z pięciu kategorii: 18-24, 25-34, 35-44, 45-54 lub 55+.

Agregacja metryk:

Dla każdej kombinacji marka + grupa wiekowa system agreguje dane, zliczając unikalnych użytkowników, sumując wyświetlenia poszczególnych typów zdarzeń (view, purchase) oraz kalkulując całkowity przychód. Współczynnik konwersji jest następnie obliczany jako stosunek liczby zakupów do liczby wyświetleń, z odpowiednią obsługą przypadku gdy liczba wyświetleń wynosi zero.

Optymalizacja: Focus na top 100 marek:

Kluczową optymalizacją jest ograniczenie wyników tylko do top 100 marek według wygenerowanego przychodu. Bez tego ograniczenia system musiałby zapisać potencjalnie tysiące rekordów do HBase (wszystkie marki pomnożone przez wszystkie grupy wiekowe), co znacząco wydłużyłoby czas wykonania i obciążałoby bazę danych niepotrzebnymi informacjami o niszowych markach.

Ta optymalizacja redukuje liczbę zapisywanych rekordów z potencjalnych 5000-10000 do około 500 (100 marek razy około 5 grup wiekowych), przy minimalnej utracie wartości biznesowej. Marki poza top 100 mają małe znaczenie strategiczne dla platformy.

8.6.3. Wyniki analizy

Jupyter notebook prezentuje kompleksowe wizualizacje efektywności marek. Analiza obejmuje 1,503 rekordów reprezentujących 100 unikalnych marek z podziałem na grupy wiekowe.

Top marki według przychodu:

Apple zdecydowanie dominuje jako lider rynku, ze znaczącym dystansem do konkurencji. Samsung zajmuje solidną drugą pozycję. Grupa pościgowa obejmuje Sony, LG i Xiaomi. Pozostałe pozycje w top 10 zajmują głównie marki elektroniczne i AGD.

Wyraźnie widoczna jest koncentracja rynku - top 10 marek generuje znaczącą większość całkowitego przychodu platformy. To typowy wzorzec w e-commerce, gdzie niewielka liczba silnych marek odpowiada za większość wartości biznesowej.

Preferencje według grup wiekowych:

Młodszy użytkownicy w wieku 18-24 lat wyraźnie preferują marki technologiczne, takie jak Apple, Samsung i Xiaomi. Ta grupa jest najbardziej zainteresowana najnowszymi smartfonami, laptopami i gadżetami elektronicznymi.

Starszy użytkownicy (45+) częściej wybierają marki AGD i narzędzi, takie jak Bosch i Philips. Ich zakupy koncentrują się bardziej na produktach praktycznych dla domu niż na najnowszych technologiach.

Grupa 25-34 lat generuje najwyższy przychód we wszystkich kategoriach marek. To najważniejszy segment demograficzny dla platformy - użytkownicy w tym wieku mają już ustabilizowaną sytuację finansową, ale wciąż są otwarci na nowe produkty i technologie.

Analiza współczynników konwersji:

Premium brands, takie jak Apple i Sony, charakteryzują się niższymi współczynnikami konwersji, ale wyższymi wartościami koszyka. Użytkownicy przeglądają wiele produktów tych marek przed podjęciem decyzji o zakupie, ale ostatecznie kupują drogie produkty, generując wysoki przychód na transakcję.

Mid-tier brands, takie jak Xiaomi i Realme, mają wyższe conversion rates, ale niższe średnie przychody. Ich produkty są bardziej przystępne cenowo, co przekłada się na szybsze decyzje zakupowe, ale mniejsze wartości zamówień.

Niektóre marki wykazują bardzo niskie conversion rates pomimo wysokich liczb wyświetleń, co sugeruje problemy z propozycją wartości lub prezentacją produktów. To sygnał ostrzegawczy dla działów handlowych i marketingowych o konieczności przeglądu współpracy z tymi markami.

8.6.4. Zapis wyników do HBase

Wyniki są zapisywane do tabeli brand_performance z wykorzystaniem dwóch grup kolumn (column families):

Column family: metrics - zawiera wszystkie metryki liczbowe: liczbę unikalnych użytkowników, sumę wyświetleń, sumę zakupów, całkowity przychód oraz współczynnik konwersji.

Column family: trends - przechowuje metadane: nazwę marki, grupę wiekową oraz timestamp utworzenia rekordu.

Klucz wiersza ma format timestamp_brand_age_group, na przykład 20260102_051700_apple_25-34. Taka struktura umożliwia efektywne wyszukiwanie rekordów według czasu, marki oraz grupy wiekowej.

System zapisuje rekordy pojedynczo, z okresowym logowaniem postępu co 50 rekordów. To pozwala na monitoring długotrwałych operacji zapisu oraz ułatwia debugowanie w przypadku błędów - administrator może obserwować postęp w logach i szybko zidentyfikować moment wystąpienia ewentualnego problemu.

8.7. Podsumowanie warstwy wsadowej

Batch Layer systemu analitycznego realizuje trzy kluczowe analizy dostarczające wartości biznesowej:

Funnel analysis identyfikuje wąskie gardła w procesie zakupowym, pokazując dokładnie gdzie użytkownicy porzucają proces i umożliwiając optymalizację współczynników konwersji poprzez celowane interwencje marketingowe i UX.

Demographic recommendations personalizuje doświadczenia zakupowe według profilu użytkownika, zwiększając prawdopodobieństwo zakupu poprzez rekomendowanie produktów popularnych w danym segmencie demograficznym.

Brand performance monitoruje efektywność marek w różnych segmentach demograficznych, wspierając strategiczne decyzje dotyczące współpracy handlowej, alokacji budżetu reklamowego oraz targetowania kampanii marketingowych.

Wszystkie analizy są w pełni zautomatyzowane, wykonując się cyklicznie według harmonogramu Airflow bez konieczności ręcznej interwencji. Wyniki są przechowywane w HBase, zapewniając dostęp dla aplikacji końcowych oraz dashboardów analitycznych.

Architektura Lambda łącząca Speed Layer z Batch Layer umożliwia równowagę między szybkością reakcji (metryki w czasie rzeczywistym dostępne w ciągu sekund) a głębią analiz (złożone metryki historyczne obliczane na większych zbiorach danych). System jest

skalowalny i może rosnąć wraz z platformą e-commerce, stanowiąc fundament dla przyszłych rozszerzeń o dodatkowe algorytmy analityczne, takie jak zaawansowane systemy rekomendacji oparte na machine learning czy predykcję zachowań użytkowników.